Содержание

Введение **4**

1 Теоретические основы и анализ архитектур детекции дисфлуенций **7**

1.1 Клинико-акустическая классификация заикания 7

1.2 Обзор существующих наборов данных и проблема доменного сдвига 8

1.3 Математический аппарат самообучающихся акустических трансформеров 9

2 Методология, инженерия данных и аппаратные оптимизации **11**

2.1 Конвейер подготовки корпуса SEP-28k и разведочный анализ 11

2.2 Математическое обоснование фокальной функции потерь . . 12

2.3 Офлайн-экстракция признаков и оптимизация ОЗУ методом квантования 12

2.4 Математическая модель физической кинематики лица 14

3 Архитектура нейросетевых классификаторов и мультимодальное слияние **17**

3.1 Базовый полносвязный классификатор 17

3.2 Временная сверточная сеть 17

3.3 Гибридная архитектура TCN с механизмом многоголового самовнимания 18

3.4 Мультимодальная архитектура раннего слияния 19

4 Вычислительные эксперименты, метрики и анализ результатов **21**

4.1 Система метрик и проблема коллапса при оптимизации F1-меры 21

4.2 Сравнительный анализ акустических и гибридных архитектур на корпусе SEP-28k 22

4.3 Формирование русскоязычного видеокорпуса и протокол валидации 24

Заключение **26**

Список литературы **28**

Приложение А. Полные листинги программного кода **29**

Введение

В современном цифровом обществе системы автоматического распознавания речи, голосовые интерфейсы, средства телемедицины и интеллектуальные диалоговые ассистенты стали неотъемлемой частью технологического ландшафта [3, 5]. Однако для значительной доли населения, страдающей патологическими нарушениями беглости речи, эти технологии зачастую остаются малодоступными. Заикание представляет собой сложное нейромоторное нарушение речи, которое характеризуется непроизвольными спазмами артикуляционного аппарата, приводящими к внезапным остановкам (блокам), многократным повторениям отдельных звуков, слогов или целых слов, а также неестественным удлинениям (пролонгациям) фонем [2]. Стандартные акустические модели распознавания речи, обученные преимущественно на корпусах плавной дикторской речи, воспринимают подобные дисфлуенции как фоновый акустический шум, конец высказывания или неразборчивую артикуляцию, что приводит к критическим сбоям в транскрибации и семантическом анализе.

Разработка алгоритмов автоматической детекции и классификации заиканий является актуальной междисциплинарной задачей на стыке вычислительной лингвистики, цифровой обработки сигналов, компьютерного зрения и глубокого машинного обучения [1, 2]. Существующие автоматизированные детекторы, опирающиеся исключительно на акустическую модальность, сталкиваются с фундаментальным противоречием между полнотой (Recall) и специфичностью (Specificity). Модели, настроенные на высокую чувствительность, успешно находят большинство патологических эпизодов, но одновременно демонстрируют крайне низкую специфичность, ошибочно маркируя естественные интонационные паузы, вдохи, смысловые акценты и раздумья как заикание [1]. Подобная гипердиагностика делает сгенерированные временные шкалы непригодными для клинического анализа логопедами и исследователями [1].

Второй фундаментальной проблемой является языковая зависимость существующих решений. Подавляющее большинство современных открытых наборов данных, таких как SEP-28k и FluencyBank, содержат исключительно англоязычную речь [2]. Акустические трансформеры, дообучен-

ные на этих ресурсах, склонны переобучаться на специфические фонетические, интонационные и просодические паттерны английского языка [1]. При попытке прямого переноса таких моделей на русскоязычные аудио- и видеозаписи возникает выраженный эффект доменного сдвига, ведущий к резкой деградации обобщающей способности классификатора [1].

Целью данной производственной практики является комплексное исследование, математическое обоснование и программная реализация робастной мультимодальной системы автоматической детекции заиканий, объединяющей пофреймовый акустический анализ с физической кинематикой лица и способной к эффективному межъязыковому переносу в условиях строгих аппаратных ограничений персональных вычислительных систем.

Для достижения поставленной цели в ходе работы были сформулированы и решены следующие научно-практические задачи:

1. проведение аналитического обзора предметной области, исследование существующих корпусов дисфлуенций [2], анализ уязвимостей SOTA-архитектур: Vocametrix, StutterLens [1] и формирование строгого математического аппарата оценки качества классификации;

2. программная реализация автоматизированного конвейера загрузки, декодирования, ресемплинга и стандартизации крупномасштабного набора данных SEP-28k (31 908 аудиофрагментов) [2] с проведением разведочного анализа данных;

3. разработка модулей офлайн-экстракции пофреймовых признаков с использованием трех предобученных самообучающихся трансформеров: Wav2Vec2-Large-XLSR-53 [3], WavLM-Base-Plus [4], Whisper-Small [5] и внедрение алгоритмов низкоуровневого квантования памяти (float16), позволивших сократить потребление ОЗУ с 19.6 ГБ до 9.8 ГБ;

4. математическое обоснование и реализация архитектур временных сверточных сетей на базе каскада расширенных остаточных сверток [7], а также гибридной архитектуры с механизмом многоголового самовнимания (Multi-Head Self-Attention) и маскированного глобального усреднения;

5. выявление и устранение проблемы коллапса мажоритарного класса при оптимизации по F1-мере на несбалансированных выборках путем перевода критерия ранней остановки (Early Stopping) и сохранения весов на интегральную метрику ранжирования ROC-AUC в сочетании с фокальной функцией потерь [8];

6. проектирование визуального контура извлечения 42 языконезависимых параметров физической кинематики лица на базе трехмерной сетки MediaPipe Face Mesh [6] с расчетом скоростей, ускорений, индекса тремора и робастной нормализацией по межквартильному размаху (IQR) [1];

7. разработка архитектуры глубокого раннего слияния модальностей [7] с механизмом модального дропаута и реализация конвейера скользящего окна для кросс-языковой валидации на русскоязычных видеозаписях [1].

1. Теоретические основы и анализ архитектур детекции дисфлуенций

1.1. Клинико-акустическая классификация заикания

С точки зрения речевой патологии и цифровой обработки сигналов, проявления заикания обладают высокой гетерогенностью и затрагивают как спектрально-временные характеристики звука, так и биомеханику лицевой мускулатуры [1, 2]. В рамках компьютерного моделирования выделяются пять основных типов событий, представленных в разметке современных датасетов [2]:

1. блоки: непроизвольные внезапные остановки речевого потока, вызванные тоническим спазмом мышц гортани, языка или губ; на спектрограмме блок проявляется как участок полной или почти полной тишины, которому предшествует или за которым следует резкий выброс акустической энергии; исключительно по аудиосигналу блок крайне сложно отличить от обычной смысловой паузы, однако визуально он сопровождается выраженным напряжением губ, асимметрией челюсти или тремором [1];

2. пролонгации: неестественное растягивание отдельных звуков (чаще согласных фрикативных или сонорных фонем) во времени; в спектральной области пролонгация выглядит как квазистационарный участок с неизменными формантными частотами, длительность которого существенно превышает статистическую норму для данного темпа речи [2];

3. повторения звуков и слогов: клонические спазмы, приводящие к многократному циклическому воспроизведению начального звука или слога слова; формируют характерный высокочастотный ритмический паттерн модуляций огибающей сигнала [2];

4. повторения слов: неоднократное повторение целых односложных или многосложных слов, возникающее как попытка говорящего преодолеть речевой ступор [2];

5. интеръекции: вводные паразитные звуки и заполненные паузы («э-э», «м-м», «ну»), используемые для маскировки спазма или отсрочки произнесения трудной фонемы [2].

Разнородность указанных классов делает неэффективным применение классических линейных моделей или простых спектральных пороговых детекторов, требуя построения иерархических моделей, способных одновременно анализировать локальную микродинамику и глобальный контекст окна [7].

1.2. Обзор существующих наборов данных и проблема доменного сдвига

Ключевым индустриальным стандартом для обучения систем детекции речевых нарушений выступает корпус SEP-28k (Stuttering Events in Podcasts), разработанный исследователями компании Apple и дополненный записями из корпуса FluencyBank [2]. Объединенный набор содержит более 31 000 трехсекундных аудиоклипов, извлеченных из англоязычных подкастов с участием людей с заиканием [2]. Среди других известных ресурсов выделяются синтетический корпус LibriStutter и немецкоязычный терапевтический корпус KSOF (Kassel State of Fluency) [1, 2].

Несмотря на большой объем SEP-28k, модели, обученные исключительно на нем, страдают от сильного доменного сдвига при переносе на другие языки [1]. В исследовании Е. Иванкина и Н. Базенкова [1] была сформирована пилотная база из 39 русскоязычных видеозаписей от 20 дикторов с заиканием, размеченная экспертами с точностью до кадра. Статистический анализ этого корпуса показал, что медианная длительность эпизодов заикания в живой русской речи составляет менее 1.0 секунды [1]. Это означает, что модели, усредняющие признаки на 3-секундном интервале, неизбежно размывают короткие патологические события на фоне окружающих их нормальных слов.

При тестировании открытых англоязычных аудиомоделей на русскоязычных записях авторами были зафиксированы следующие показатели [1]:

1. архитектура Vocametrix (построенная на базе дообученного транс-

формера wav2vec2-large-xlsr-53 [3]) продемонстрировала полноту Recall = 0.957, но критически низкую точность Precision = 0.327 и специфичность Specificity = 0.300 (F1 = 0.487, ROC-AUC = 0.701) [1]; из-за отличий в русской интонации и фонетике модель классифицировала 70% нормальной речи как заикание;

2. архитектура StutterLens показала обратный перекос: высокую специфичность Specificity = 0.886, но крайне низкую полноту Recall = 0.205 (Precision = 0.391, F1 = 0.269, ROC-AUC = 0.632), пропуская четыре из пяти реальных спазмов [1];

3. предложенная в лабораторном исследовании схема позднего слияния, объединяющая выходы Vocametrix с 20 лицевыми единицами действия (Facial Action Units, FAU из пакета PyFeat) через классификатор Histogram-based Gradient Boosting с ограничением на высокую специфичность, позволила довести Specificity до 0.929 (Precision = 0.504, ROC-AUC = 0.710), но обрушила полноту до Recall = 0.195 (F1 = 0.282) [1].

Причиной падения полноты в схеме Late Fusion является то, что абстрактные эмоциональные единицы FAU, предназначенные для распознавания улыбки, гнева или удивления, слабо коррелируют с высокочастотной биомеханикой речевого спазма, а объединение на уровне готовых вероятностей не позволяет сети находить нелинейные кросс-модальные связи [1].

1.3. Математический аппарат самообучающихся акустических трансформеров

Для преодоления фонетической зависимости в настоящей работе проводится систематическое исследование трех фундаментальных акустических архитектур, различающихся парадигмой предобучения [3, 4, 5]:

1. Wav2Vec2-Large-XLSR-53 [3]: модель принимает на вход нормированный сырой аудиосигнал $X = (x_1, \ldots, x_N)$ с частотой дискретизации 16 кГц; многослойный сверточный энкодер $f : \mathcal{X} \mapsto \mathcal{Z}$ с шагом свертки 320 отсчетов, что соответствует временному разрешению 20 мс или частоте 50 Гц, преобразует сигнал в последовательность латентных векторов $z_1, \ldots, z_T$;

далее векторы подаются в 24-слойный блок Transformer $g : \mathcal{Z} \mapsto \mathcal{C}$, формирующий контекстные репрезентации $c_t \in \mathbb{R}^{1024}$; в процессе самообучения на 53 языках часть векторов z_t маскируется, и модель минимизирует контрастивную функцию потерь, угадывая истинный квантованный вектор q_t среди набора негативных дистракторов $\tilde{q} \in \mathcal{Q}_t$ [3]:

$$\mathcal{L}_m = -\log \frac{\exp(\mathrm{sim}(c_t, q_t)/\kappa)}{\sum_{\tilde{q}\in\mathcal{Q}_t} \exp(\mathrm{sim}(c_t, \tilde{q})/\kappa)}, \quad \mathrm{sim}(a, b) = \frac{a^\top b}{\|a\|\|b\|} \tag{1}$$

где κ — температурный гиперпараметр; несмотря на мультиязычность, целевая функция Wav2Vec2 ориентирована на различение фонем, что делает ее чувствительной к артикуляционным сдвигам [3];

2. WavLM-Base-Plus [4]: архитектура от Microsoft расширяет концепцию самообучения за счет механизма Masked Speech Denoising [4]; во время предобучения к входным высказываниям искусственно добавляются перекрывающиеся голоса других дикторов и акустические шумы, однако от трансформера требуется предсказать дискретные псевдо-метки чистого целевого сигнала на замаскированных позициях; кроме того, в механизме самовнимания используется относительное позиционное смещение [4]; благодаря этому 12-слойный энкодер (hidden_dim = 768) сохраняет в скрытых состояниях богатую паралингвистическую информацию: тембр, микротремор голосовых связок, дыхательные паузы и акустические артефакты мышечного зажима [4];

3. Whisper-Small [5]: модель от OpenAI обучена в режиме слабой супервизии на беспрецедентном объеме данных — 680 000 часов многоязычного и мультизадачного аудио [5]; в отличие от Wav2Vec2 и WavLM, на вход Whisper подается 80-канальная лог-мел спектрограмма, вычисленная в окнах по 25 мс с шагом 10 мс; два входных сверточных слоя с функцией активации GELU и шагом 2 сжимают временную ось в два раза (до разрешения 20 мс, 50 фреймов в секунду), после чего тензор обрабатывается стеком блоков Transformer (hidden_dim = 768) [5]; поскольку модель обучалась одновременно распознавать речь, переводить ее и детектировать активность голоса на десятках языков, ее энкодер формирует инвариантные к языку репрезентации просодии и ритмики [5].

2. Методология, инженерия данных и аппаратные оптимизации

2.1. Конвейер подготовки корпуса SEP-28k и разведочный анализ

На первом этапе работы был разработан модуль автоматизированной загрузки и подготовки данных sep28k_data_preparation.py [2]. Для обхода системных ошибок декодирования аудиокодеков на платформе Windows загрузка из репозитория HuggingFace (sep28k-fluencybank-stutter-dataset [2]) осуществлялась в режиме чтения сырых байтовых потоков. Каждый фрагмент декодировался в оперативной памяти через io.BytesIO и библиотеку soundfile. Стереофонические записи приводились к монофоническому виду путем усреднения каналов:

$$x_{\text{mono}}[n] = \frac{1}{C} \sum_{c=1}^{C} x_c[n] \tag{2}$$

При отклонении исходной частоты дискретизации f_s от целевого стандарта 16 000 Гц выполнялась полифазная передискретизация через функцию librosa.resample. Бинарная целевая метка is_stutter присваивалась равной 1, если хотя бы один из клинических индикаторов: Prolongation, Block, SoundRep, WordRep принимал положительное значение [2]. Всего было успешно стандартизировано и сохранено на диск 31 908 аудиофайлов в формате WAV и сформирован реестр metadata.csv.

С помощью разработанного скрипта sep28k_data_analysis.py был проведен разведочный анализ данных. Анализ выявил существенный дисбаланс классов [2]:

1. фрагменты плавной речи (класс 0): 9 579 файлов (30.02%);
2. фрагменты с наличием дисфлуенций (класс 1): 22 329 файлов (69.98%).

Среди отдельных типов нарушений зафиксировано значительное перекрытие (в одном 3-секундном окне может присутствовать как блок, так и последующее повторение звука) [2].

2.2. Математическое обоснование фокальной функции потерь

При соотношении классов 70:30 стандартная бинарная кросс-энтропия генерирует доминирующий градиент от мажоритарного позитивного класса, побуждая классификатор предсказывать наличие заикания для всех входных примеров [8]. Для решения этой проблемы во все нейросетевые модели была внедрена фокальная функция потерь [8].

Пусть $\hat{y} \in \mathbb{R}$ — выходной логит нейросети, а $p = \sigma(\hat{y}) = (1 + e^{-\hat{y}})^{-1}$ — оцененная вероятность принадлежности к классу 1. Обозначим вероятность истинного класса $y \in \{0, 1\}$ как p_t:

$$p_t = \begin{cases} p, & \text{если } y = 1 \\ 1 - p, & \text{если } y = 0 \end{cases} \tag{3}$$

Тогда классическая кросс-энтропия записывается как $\mathrm{BCE}(p_t) = -\log(p_t)$, а фокальная функция потерь вводит модулирующий множитель $(1 - p_t)^\gamma$ и балансирующий вес α [8]:

$$\mathcal{L}_{\mathrm{Focal}}(p_t) = -\alpha(1 - p_t)^\gamma \log(p_t) = \alpha \left(1 - e^{-\mathrm{BCE}}\right)^\gamma \cdot \mathrm{BCE} \tag{4}$$

При выборе параметров $\gamma = 2.0$ и $\alpha = 0.25$ примеры, в которых сеть уже уверена ($p_t \to 1$), получают близкий к нулю вес $(1 - p_t)^2 \to 0$, тогда как сложные и ошибочно классифицируемые примеры негативного класса сохраняют высокий градиент на протяжении всего цикла обучения [8].

2.3. Офлайн-экстракция признаков и оптимизация ОЗУ методом квантования

Полноценное дообучение крупных трансформеров одновременно с классификатором требует хранения промежуточных активаций всех 12–24 слоев для вычисления обратного распространения ошибки, что потребляет свыше 16–24 ГБ видеопамяти [3, 4]. Поскольку вычислительный стенд оснащен мобильным графическим ускорителем NVIDIA GeForce RTX 4050 (6 ГБ VRAM) и 16 ГБ системной оперативной памяти, была спроектирована двухэтапная схема с офлайн-экстракцией признаков.

Веса предобученных трансформеров замораживались, модели перево-

дились в режим инференса, и каждый аудиофайл однократно пропускался через энкодер в контексте. В отличие от базового подхода: extract_features.py, где матрица скрытых состояний $H \in \mathbb{R}^{T \times D}$ сжималась вдоль оси времени в один вектор $v = \frac{1}{T} \sum_{t=1}^{T} H_t \in \mathbb{R}^D$, в секвенциальных скриптах: extract_features_seq.py, extract_features_wavlm.py, extract_features_whisper.py на диск сохранялась полная двумерная матрица размерности (T, D), где $T \approx 150$ фреймов для 3-секундной записи [3, 4, 5].

Особую алгоритмическую задачу представляла экстракция признаков из модели Whisper-Small [5]. Архитектура Whisper жестко требует на вход аудиосигнал длительностью ровно 30 секунд (480 000 отсчетов), формируя на выходе энкодера тензор фиксированной длины $T_{\max} = 1500$ фреймов [5]. Сохранение 1500 фреймов для 3-секундного клипа привело бы к 90% избыточности, сохранению представлений нулевого паддинга. Поэтому в скрипте extract_features_whisper.py было реализовано динамическое вычисление реального числа информативных фреймов на основе истинной длительности сигнала L_{samples} и общего шага свертки (stride = 320):

$$T_{\text{real}} = \min\left(\left\lceil \frac{L_{\text{samples}}}{320} \right\rceil, 1500\right) \tag{5}$$

После чего на диск сохранялся усеченный срез hidden_states[0, :real_frames, :].

Разрешение проблемы переполнения ОЗУ: Для обеспечения высокой скорости обучения сверточных сетей (время эпохи 23–45 секунд) все 31 908 матриц должны кэшироваться в оперативную память перед началом эпох, чтобы исключить "узкое горлышко"дискового ввода-вывода. Однако при стандартной одинарной точности float32 (4 байта на элемент) суммарный объем памяти для матриц Wav2Vec2 ($T = 150, D = 1024$) составляет:

$$V_{\text{fp32}} = 31\,908 \times 150 \times 1024 \times 4 \text{ байт} = 19\,604\,275\,200 \text{ байт} \approx 19.60 \text{ ГБ} \tag{6}$$

На рабочей станции с 16 ГБ ОЗУ это неизбежно вызывало критическую ошибку нехватки памяти. Для решения проблемы в классах SequentialDataset и MeanPooledDataset (модуль utils.py) было внедрено хранение массивов в формате половинной точности IEEE 754 float16 (2

байта на элемент):

$$V_{\text{fp16}} = 31\,908 \times 150 \times 1024 \times 2 \text{ байт} \approx 9.80 \text{ ГБ} \tag{7}$$

При запросе конкретного сэмпла загрузчиком DataLoader в методе __getitem__ выполняется мгновенное приведение типа к torch.float32 непосредственно перед отправкой батча в видеопамять. Поскольку выходные активации нормализованных слоев трансформеров лежат в узком динамическом диапазоне $[-10, 10]$, переход к float16 при хранении не вносит численной погрешности в обучение, но сокращает потребление ОЗУ ровно в два раза.

2.4. Математическая модель физической кинематики лица

Для замены нерелевантных эмоциональных признаков FAU [1] был разработан модуль extract_face_kinematics.py, использующий нейросетевой детектор MediaPipe Face Mesh [6] для извлечения трехмерных координат $P_i = (x_i, y_i, z_i)$ 478 анатомических точек лица с целевой частотой 15 кадров в секунду.

Чтобы обеспечить инвариантность признаков к расстоянию от человека до камеры и разрешению видео, в каждом кадре вычисляется базовый масштаб лица S_{face} как евклидово расстояние в $\mathbb{R}^3$ между переносицей ($idx = 6$) и кончиком носа ($idx = 1$) [6]:

$$S_{\text{face}} = \|P_6 - P_1\|_2 = \sqrt{(x_6 - x_1)^2 + (y_6 - y_1)^2 + (z_6 - z_1)^2} \tag{8}$$

На основе координат ключевых точек рассчитываются 7 базовых биомеханических параметров кадра:

1. коэффициент раскрытия рта: отношение среднего вертикального расстояния между тремя парами внутренних точек верхней ($U = \{82, 13, 312\}$) и нижней ($L = \{87, 14, 317\}$) губы к горизонтальному расстоянию между уголками рта (61 и 291):

$$\text{MAR} = \frac{\frac{1}{3}\sum_{k=1}^{3} \|P_{U_k} - P_{L_k}\|_2}{\max(\|P_{61} - P_{291}\|_2, 10^{-6})} \tag{9}$$

;

2. растяжение уголков губ: $\mathrm{LCS} = \|P_{61} - P_{291}\|_2 / S_{\text{face}}$;

3. вертикальное смещение челюсти: нормированная координата подбородка ($idx = 152$): $\mathrm{JD} = y_{152} / S_{\text{face}}$;

4. напряжение бровей: нормированное сближение внутренних краев бровей (70 и 300): $\mathrm{BT} = \|P_{70} - P_{300}\|_2 / S_{\text{face}}$;

5. высота бровей: среднее вертикальное расстояние от бровей до верхних век глаз (159 и 386):

$$\mathrm{BH} = \frac{(y_{70} - y_{159}) + (y_{300} - y_{386})}{2 S_{\text{face}}} \tag{10}$$

;

6. апертура глаз: среднее раскрытие левого $(159, 145)$ и правого $(386, 374)$ глаза, фиксирующее сопутствующее заиканию зажмуривание и тики:

$$\mathrm{EA} = \frac{\|P_{159} - P_{145}\|_2 + \|P_{386} - P_{374}\|_2}{2 S_{\text{face}}} \tag{11}$$

;

7. латеральная асимметрия челюсти: разница расстояний от левого (234) и правого (454) краев нижней челюсти до подбородка (152), выявляющая боковые спазмы:

$$\mathrm{JA} = \frac{\left| \|P_{234} - P_{152}\|_2 - \|P_{454} - P_{152}\|_2 \right|}{S_{\text{face}}} \tag{12}$$

.

Кадры, на которых лицо кратковременно не было обнаружено детектором, восстанавливаются методом покоординатной линейной интерполяции [1]. Далее для каждого видеоролика индивидуально выполняется робастная нормализация каждого признака f_j по медиане и межквартильному размаху: $\mathrm{IQR} = Q_{75} - Q_{25}$ с усечением выбросов на уровне $[-6, 6]$ [1]:

$$\tilde{f}_j(t) = \mathrm{clip}\left(\frac{f_j(t) - \mathrm{median}(f_j)}{Q_{75}(f_j) - Q_{25}(f_j)}, -6, 6 \right) \tag{13}$$

Это полностью устраняет влияние индивидуальных анатомических пропорций лица конкретного пациента [1].

На заключительном этапе для каждого из 7 нормированных признаков при шаге дискретизации $\Delta t = 1/\mathrm{FPS}$ вычисляются 5 динамических характеристик:

1. первая производная (скорость): $v_j(t) = (\tilde{f}_j(t) - \tilde{f}_j(t-1))/\Delta t$;
2. вторая производная (ускорение): $a_j(t) = (v_j(t) - v_j(t-1))/\Delta t$;
3. абсолютная скорость: $|v_j(t)|$;
4. скользящая кинетическая энергия в окне W длительностью 0.3 с: $E_j(t) = \frac{1}{|W|}\sum_{\tau \in W(t)} |v_j(\tau)|$;
5. индекс мышечного тремора (стандартное отклонение скорости в окне 0.3 с):

$$\mathrm{Tremor}_j(t) = \sqrt{\frac{1}{|W|}\sum_{\tau \in W(t)} \big(v_j(\tau) - \bar{v}_j(t)\big)^2} \tag{14}$$

.

Конкатенация базовых значений и пяти производных формирует итоговый пофреймовый вектор размерности $7 \times 6 = 42$ признака.

3. Архитектура нейросетевых классификаторов и мультимодальное слияние

3.1. Базовый полносвязный классификатор

В качестве отправной точки исследования (Baseline) в скрипте train_mlp.py была реализована трехслойная полносвязная сеть AudioClassifier, принимающая на вход усредненный по времени вектор размерности $D = 1024$ [3]. Архитектура описывается последовательностью преобразований:

$$x \in \mathbb{R}^{1024} \xrightarrow{\text{Linear+BN+ReLU+Drop(0.4)}} \mathbb{R}^{512} \xrightarrow{\text{Linear+BN+ReLU+Drop(0.3)}} \mathbb{R}^{128} \xrightarrow{\text{Linear}} \mathbb{R}^{1} \tag{15}$$

Обучение проводилось оптимизатором AdamW ($\text{lr} = 10^{-4}$, $\text{weight_decay} = 10^{-3}$) с функцией потерь Focal Loss [8] и адаптивным планировщиком ReduceLROnPlateau, уменьшающим шаг обучения в 2 раза при отсутствии роста валидационного ROC-AUC в течение 2 эпох.

3.2. Временная сверточная сеть

Для сохранения временной структуры сигнала в модуле train_tcn.py была разработана архитектура StutterTCN, основанная на принципах построения временных сверточных сетей [7]. Входная матрица $X \in \mathbb{R}^{B \times T \times D}$ транспонируется к виду (B, D, T) и проходит через точечную свертку с батч-нормализацией и активацией ReLU, проецируясь в пространство размерности $H = 256$.

Ядром сети выступает стек из $K = 4$ остаточных блоков ResidualBlock с экспоненциально растущим коэффициентом дилатации $d_i = 2^i \in \{1, 2, 4, 8\}$ и размером ядра свертки $k = 5$ [7]. Внутри каждого блока одномерная расширенная свертка определяется как [7]:

$$y[t] = \sum_{m=0}^{k-1} w[m] \cdot x\left[t + d \cdot (m - \lfloor k/2 \rfloor)\right] \tag{16}$$

За счет симметричного паддинга $p = (k-1) \cdot d/2$ длина последовательности T сохраняется неизменной, а выход блока вычисляется с остаточной связью

[7]:

$$Z_{\text{out}} = \text{ReLU}\Big(Z_{\text{in}} + \text{BN}_2\big(\text{Conv}_{1\times1}(\text{Dropout}(\text{ReLU}(\text{BN}_1(\text{Conv}_{k,d}(Z_{\text{in}})))))\big)\Big) \tag{17}$$

Эффективное рецептивное поле стека из 4 блоков при $k = 5$ составляет [7]:

$$\text{RF} = 1 + (k-1)\sum_{i=0}^{3} 2^i = 1 + 4 \times (1+2+4+8) = 61 \text{ фрейм} \tag{18}$$

При разрешении 20 мс на фрейм один нейрон верхнего слоя охватывает контекст шириной 1.22 секунды, что идеально соответствует характерной длительности речевых блоков и пролонгаций [1, 2].

Для агрегации признаков переменной длины применяется маскированное глобальное усреднение. Если клип короче максимальной длины $T_{\max} = 150$, бинарная маска $M \in \{0,1\}^{B\times T}$ обнуляет вклад паддинга:

$$c = \frac{\sum_{t=1}^{T_{\max}} H_t \odot M_t}{\max\left(\sum_{t=1}^{T_{\max}} M_t, 1\right)} \in \mathbb{R}^{256} \tag{19}$$

Финальный вектор c классифицируется двухслойной головой $256 \to 128 \to 1$.

3.3. Гибридная архитектура TCN с механизмом многоголового самовнимания

Несмотря на эффективность расширенных сверток при поиске локальных аномалий, операция простого усреднения всех T фреймов на выходе TCN может размывать сигнал коротких спазмов длительностью 0.2–0.4 с на фоне окружающей плавной речи. Кроме того, повторения целых слов требуют учета глобальных зависимостей между началом и концом 3-секундного окна.

Для решения этой задачи в модуле train_hybrid_tcn.py была спроектирована гибридная архитектура StutterHybridTCN, объединяющая сверточный стек TCN с трансформерным слоем многоголового самовнимания: Multi-Head Self-Attention, MHA, число голов $h = 4$, размерность $d_{\text{model}} = 256$. После прохождения 4 блоков TCN тензор $Z \in \mathbb{R}^{B\times T\times 256}$ по-

дается в блок MHA, где для каждой головы $i \in \{1, \ldots, 4\}$ вычисляются матрицы запросов $Q_i = ZW_i^Q$, ключей $K_i = ZW_i^K$ и значений $V_i = ZW_i^V$ ($d_k = d_{\text{model}}/h = 64$):

$$\text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}} + \tilde{M}\right) V_i \tag{20}$$

где маска паддинга $\tilde{M}$ принимает значение $-\infty$ на невалидных фреймах, полностью исключая их из распределения внимания. Выход слоя внимания складывается с выходом TCN через остаточную связь и нормализуется слоем LayerNorm:

$$Z_{\text{hybrid}} = \text{LayerNorm}\big(Z + \text{MHA}(Z, Z, Z)\big) \tag{21}$$

после чего выполняется маскированное глобальное усреднение и финальная классификация.

3.4. Мультимодальная архитектура раннего слияния

В модуле train_deep_fusion.py реализована архитектура DeepFusionTCN, объединяющая аудиопоток $X_a \in \mathbb{R}^{B\times T\times D_a}$ и синхронизированный поток лицевой кинематики $X_v \in \mathbb{R}^{B\times T\times 42}$ до этапа классификации [1, 7]. Поскольку частота кадров видео (15 Гц) и фреймов аудио (50 Гц) различаются, в классе MultimodalDataset выполняется линейная интерполяция более короткого ряда к общей временной сетке $T_{\text{target}} = \max(T_a, T_v)$, после чего последовательности нарезаются скользящим окном длиной 450 отсчетов с шагом 150. Окно получает позитивную метку, если доля размеченных заиканием фреймов внутри него составляет не менее 30%.

Аудиопоток кодируется 4-блочным энкодером StreamEncoder ($d \in \{1, 2, 4, 8\}$) в тензор $F_a \in \mathbb{R}^{B\times 256\times T}$, а видеопоток — 3-блочным энкодером ($d \in \{1, 2, 4\}$) в тензор $F_v \in \mathbb{R}^{B\times 256\times T}$ [7].

Механизм модального дропаута: чтобы предотвратить переобучение сети исключительно на более размерный акустический канал, во время тренировки с вероятностью $p_{\text{mod}} = 0.2$ генерируется случайная величина

$r \sim \mathcal{U}(0,1)$, и один из потоков принудительно зануляется:

$$(\tilde{F}_a, \tilde{F}_v) = \begin{cases} (\mathbf{0}, F_v), & \text{если } r < 0.1 \\ (F_a, \mathbf{0}), & \text{если } 0.1 \le r < 0.2 \\ (F_a, F_v), & \text{если } r \ge 0.2 \end{cases} \tag{22}$$

После этого представления конкатенируются вдоль канальной оси в тензор $F_{\text{cat}} = [\tilde{F}_a; \tilde{F}_v] \in \mathbb{R}^{B \times 512 \times T}$, сжимаются проекцией $512 \to 256$ и обрабатываются двумя совместными блоками fusion_blocks, которые выявляют кросс-модальные корреляции, например, одновременное наступление акустической тишины и всплеск тремора нижней челюсти. Для честной оценки на видеоданных как в train_face_tcn.py, так и в train_deep_fusion.py применяется групповая кросс-валидация GroupKFold по идентификаторам пациентов, исключающая попадание одного и того же диктора в обучающую и валидационную выборки [1]. Для комплексного исследования межъязыкового переноса обучение мультимодальной сети выполняется независимо для каждого из трех акустических энкодеров с сохранением специализированных чекпоинтов: best_deep_fusion_wav2vec2.pth, best_deep_fusion_wavlm.pth и best_deep_fusion_whisper.pth.

4. Вычислительные эксперименты, метрики и анализ результатов

4.1. Система метрик и проблема коллапса при оптимизации F1-меры

Для всесторонней оценки бинарных классификаторов на каждом шаге валидации вычисляется матрица ошибок: число истинно-положительных (TP), ложно-положительных (FP), истинно-отрицательных (TN) и ложно-отрицательных (FN) предсказаний, на основании которых рассчитываются [1]:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{Specificity} = \frac{TN}{TN + FP} \tag{23}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} \tag{24}$$

а также порогонезависимые метрики качества ранжирования вероятностей: площадь под рабочей характеристикой приемника: ROC-AUC и средняя точность: Average Precision (AP) [1]:

$$\text{AP} = \sum_n (\text{Recall}_n - \text{Recall}_{n-1}) \cdot \text{Precision}_n \tag{25}$$

Экспериментальное обнаружение коллапса модели по F1-мере: в первой серии запусков обучения StutterTCN на эмбеддингах Wav2Vec2 сохранение лучшего чекпоинта и условие ранней остановки опирались на максимизацию метрики F1. Поскольку доля позитивного класса в SEP-28k равна $p_{\text{pos}} \approx 0.70$ [2], тривиальный классификатор, константно выдающий ответ «1» для любого входа ($\text{Recall} = 1.0, \text{Precision} = 0.70, \text{Specificity} = 0.0$), автоматически получает:

$$\text{F1}_{\text{trivial}} = \frac{2 \times 0.70 \times 1.0}{0.70 + 1.0} = 0.8235 \tag{26}$$

В результате на 9-й эпохе первого запуска скрипт зафиксировал как «лучшую» вырожденную модель с F1 = 0.8251, у которой полнота составля-

ла 0.9906, но специфичность упала до катастрофических 0.0428 (при низком ROC-AUC = 0.6051), проигнорировав сбалансированную 16-ю эпоху (Specificity = 0.6279, ROC-AUC = 0.7845, но F1 = 0.8010).

Учитывая выводы лабораторной статьи о приоритете специфичности и недопустимости гипердиагностики [1], во всех тренировочных модулях критерий сохранения чекпоинта был заменен на максимизацию ROC-AUC. Так как константное предсказание дает ROC-AUC = 0.50, этот критерий математически гарантирует выбор модели с наилучшей разделяющей способностью между обоими классами.

4.2. Сравнительный анализ акустических и гибридных архитектур на корпусе SEP-28k

После модификации критерия чекпоинтинга было проведено полное обучение пяти конфигураций на едином стратифицированном разбиении корпуса SEP-28k (25 526 обучающих и 6 382 валидационных клипа, random_state = 42) [2]:

1. MLP (Wav2Vec2 mean-pooled) [3]: обучение в течение 40 эпох (по 2 с на эпоху); оптимум достигнут на 33-й эпохе с ROC-AUC = 0.7673;

2. TCN (Wav2Vec2) [3, 7]: число обучаемых параметров — 1 874 945; обучение продолжалось 49 эпох (сработал Early Stopping после пика на 39-й эпохе), достигнув ROC-AUC = 0.7935;

3. TCN (WavLM) [4, 7]: экстракция признаков заняла 15 мин 14 с (скорость 34.88 ф/с); число параметров TCN — 1 809 409 (при input_dim = 768); сеть сошлась значительно быстрее: пик достигнут на 7-й эпохе (ROC-AUC = 0.8143, время эпохи 23 с);

4. TCN (Whisper) [5, 7]: экстракция признаков из-за 30-секундного входного окна заняла 1 час 3 мин 31 с (8.37 ф/с); при обучении TCN уже на 1-й эпохе модель показала качественный скачок до ROC-AUC = 0.8488 и AP = 0.9308;

5. Hybrid TCN + Multi-Head Attention (Whisper) [5, 7]: обучение гибридной архитектуры StutterHybridTCN продолжалось 13 эпох (30

с на эпоху); оптимум по ранжированию достигнут на 3-й эпохе с ROC-AUC = 0.8473 и AP = 0.9304, при этом модель продемонстрировала существенно более высокую специфичность (Specificity = 0.5971) и точность (Precision = 0.8340), чем чистая сеть TCN на том же энкодере Whisper, а на 7-й эпохе специфичность гибридной модели достигала 0.6362 при Precision = 0.8429.

Сводные результаты валидации всех пяти архитектур представлены в Таблице 1.

Архитектура и энкодер	ROC-AUC	AP	F1	Precision	Recall	Specificity
MLP (Wav2Vec2 mean-pooled) [3]	0.7673	0.8779	0.8161	0.8007	0.8321	0.5172
TCN (Wav2Vec2-XLSR-53) [3, 7]	0.7935	0.8976	0.7932	**0.8447**	0.7476	**0.6795**
TCN (WavLM-Base-Plus) [4, 7]	0.8143	0.9105	0.8439	0.8060	0.8856	0.5031
TCN (Whisper-Small) [5, 7]	**0.8488**	**0.9308**	**0.8552**	0.8218	**0.8914**	0.5496
Hybrid TCN + MHA (Whisper-Small) [5, 7]	0.8473	0.9304	0.8508	0.8340	0.8683	0.5971

Таблица 1. Итоговое сравнение акустических и гибридных архитектур на валидационной выборке SEP-28k ($N = 6382$)

Полученные экспериментальные данные позволяют сформулировать четыре ключевых научных вывода:

1. превосходство секвенциального анализа над усреднением: на идентичных признаках Wav2Vec2 [3] замена полносвязного перцептрона (MLP) на временную сверточную сеть (TCN) [7] повысила специфичность с 0.5172 до 0.6795 (+16.23 п.п.), а точность — с 0.8007 до 0.8447; расширенные свертки успешно улавливают локальные границы речевых блоков, не позволяя нормальным участкам клипа маскировать патологию;

2. роль паралингвистического предобучения: переход от Wav2Vec2 [3] к WavLM [4] дал прирост ROC-AUC на +2.08 п.п. (до 0.8143) и F1-меры на +5.07 п.п. (до 0.8439), подтвердив важность деноизинга при предобучении для сохранения признаков голосового напряжения [4];

3. доминирование мультиязычного энкодера Whisper: модели на базе Whisper-Small [5] заняли лидирующие позиции по интегральному качеству разделения классов (ROC-AUC = 0.8488, AP = 0.9308), опередив базовый MLP на +8.15 п.п. по ROC-AUC; это делает Whisper оптимальным акустическим ядром для кросс-языкового переноса [1, 5];

4. повышение специфичности за счет механизма Self-Attention: внедрение слоя многоголового самовнимания поверх сверточных блоков (Hybrid TCN + MHA) при сохранении практически идентичного уровня ROC-AUC (0.8473) и AP (0.9304) позволило увеличить специфичность на признаках Whisper с 0.5496 до 0.5971 (+4.75 п.п.) и поднять точность до 0.8340; это подтверждает, что глобальное взвешивание фреймов механизмом внимания эффективно подавляет ложные тревоги на интонационных паузах.

4.3. Формирование русскоязычного видеокорпуса и протокол валидации

Для проведения межъязыкового переноса и мультимодальной валидации был сформирован специализированный тестовый корпус из 6 непрерывных видеозаписей русской речи с клиническими проявлениями заикания: блоки, пролонгации, клонические повторы. Первичный автоматизированный аудит с помощью разработанного модуля auto_inspect_videos.py выявил наличие статических заставок и логотипов длительностью 5–6 секунд в начале каждого файла, на которых лицо диктора отсутствовало.

Для исключения искажения кинематических признаков и предотвращения ложных срабатываний классификаторов на фоновой музыке заставок был применен модуль trim_intros.py. Данный модуль выполнил покадровую обрезку неинформативных начальных участков через интерфейс FFmpeg, повторное демультиплексирование звука в монофонический формат PCM 16 кГц и автоматический пересчет временных меток заикания со сдвигом временной оси.

Характеристики полученного стандартизированного русскоязычного набора данных представлены в Таблице 2.

Для каждого стандартизированного ролика модулем extract_face_kinematics.py извлечены 42 параметра кинематики лица на базе MediaPipe Face Mesh, а модулем extract_russian_all_audio.py осуществлена поблочная экстракция акустических представлений для трех исследуемых энкодеров: Wav2Vec2-Large-XLSR-53 (размерность 1024), WavLM-Base-Plus (768) и Whisper-Small (768).

Синхронизированные покадровые потоки поступают на вход модуля

Идентификатор	Чистая длительность, с	Кадры видео (15 Гц)	Фреймы аудио (50 Гц)	Доля заикания, %	Число эпизодов
ru_01	46.3	695	2316	47.5	4
ru_02	115.9	1739	5803	91.5	4
ru_03	44.9	673	2247	76.8	3
ru_04	37.0	556	1853	78.4	2
ru_05	41.9	629	2100	93.1	2
ru_06	44.9	673	2246	46.8	5
Всего	330.9 (5.5 мин)	4965	16 565	75.8 (средн.)	20

Таблица 2. Количественные характеристики подготовленного русскоязычного видеокорпуса

evaluate_crosslingual.py, реализующего алгоритм скользящего окна с усреднением перекрывающихся предсказаний:

$$P_{\text{frame}}[t] = \frac{\sum_{w:t\in[w_s,w_e)} p_w}{\max\big(\text{count}[t], 1\big)} \tag{27}$$

Структура итогового сравнительного протокола, сопоставляющего разрабатываемые модели с известными лабораторными бейзлайнами, приведена в Таблице 3.

Категория модели	Архитектура и модальности	ROC-AUC	AP	F1	Precision	Recall	Specificity
Лабораторные бейзлайны [1]	Vocametrix (Audio Wav2Vec2)	0.7010	—	0.4870	0.3270	0.9570	0.3000
Лабораторные бейзлайны [1]	StutterLens (Audio)	0.6320	—	0.2690	0.3910	0.2050	0.8860
Лабораторные бейзлайны [1]	Lab Late Fusion (Vocametrix + FAU)	0.7100	—	0.2820	0.5040	0.1950	0.9290
Одномодальные (Zero-Shot)	Audio TCN (Wav2Vec2-XLSR-53)	Ожидает финального прогона инференса					
Одномодальные (Zero-Shot)	Audio TCN (WavLM-Base-Plus)	Ожидает финального прогона инференса					
Одномодальные (Zero-Shot)	Audio TCN (Whisper-Small)	Ожидает финального прогона инференса					
Визуальная абляция	Face TCN (42 кинематических параметра)	Ожидает завершения кросс-валидации					
Мультимодальные (Early Fusion)	Deep Fusion (Wav2Vec2 + Kinematics)	Ожидает завершения кросс-валидации					
Мультимодальные (Early Fusion)	Deep Fusion (WavLM + Kinematics)	Ожидает завершения кросс-валидации					
Мультимодальные (Early Fusion)	Deep Fusion (Whisper + Kinematics)	Ожидает завершения кросс-валидации					

Таблица 3. Сводный протокол кросс-языковой и мультимодальной валидации на русской речи

Заключение

В результате выполнения производственной практики была спроектирована, математически обоснована и программно реализована полнофункциональная модульная система для автоматической мультимодальной детекции заиканий.

Основные итоги и достижения проделанной работы заключаются в следующем:

1. сформирован и стандартизирован локальный корпус из 31 908 аудиозаписей SEP-28k [2]; проведен разведочный анализ, обосновавший применение фокальной функции потерь [8];

2. разработана архитектура офлайн-экстракции пофреймовых признаков для трансформеров Wav2Vec2 [3], WavLM [4] и Whisper [5] с внедрением механизма квантования кэша ОЗУ в формат float16, что снизило потребление памяти с 19.6 ГБ до 9.8 ГБ и обеспечило стабильное обучение на системе с 16 ГБ RAM и 6 ГБ VRAM;

3. выявлен и аналитически доказан эффект коллапса мажоритарного класса при сохранении весов по F1-мере на несбалансированных данных; осуществлен переход на критерий максимизации ROC-AUC [1];

4. доказано преимущество временных сверточных сетей [7] над усреднением признаков: специфичность классификации на базе Wav2Vec2 выросла с 0.5172 до 0.6795;

5. в ходе сравнительного тестирования установлено, что наилучшим качеством разделения обладает энкодер Whisper-Small [5]: связка TCN + Whisper достигла рекордных показателей ROC-AUC = 0.8488, AP = 0.9308 и F1 = 0.8552, а гибридная архитектура Hybrid TCN + MHA (ROC-AUC = 0.8473) позволила дополнительно повысить специфичность детекции на +4.75 п.п. (до 0.5971) и точность до 0.8340;

6. сформирован и программно стандартизирован тестовый русскоязычный видеокорпус (6 роликов, суммарно 4965 видеокадров и 16 565 аудиофреймов); разработаны и верифицированы модули автоматического аудита видимости лица, устранения артефактных заставок с

пересчетом временной шкалы, экстракции 42 параметров кинематики лица и блочного извлечения акустических эмбеддингов по трем архитектурам трансформеров; подготовлена программная база для мультимодального обучения и сравнительной покадровой сегментации;

7. реализован полный программный конвейер для кросс-языковой и мультимодальной валидации на русскоязычных видеозаписях, включающий модули автоматического извлечения аудиодорожек: prepare_russian_audio.py, блочной экстракции акустических признаков для длинных записей по трем трансформерам: extract_russian_all_audio.py, расчета 42 параметров физической кинематики лица: MediaPipe Face Mesh [6], синхронизации таймингов разметки: create_labels.py, раннего глубокого слияния с модальным дропаутом для всех трех энкодеров: train_deep_fusion.py [7] и покадровой сегментации скользящим окном: evaluate_crosslingual.py [1].

Созданная кодовая база полностью готова к финальному развертыванию и тестированию на русскоязычных видеозаписях для клинической поддержки принятия решений [1].

Список литературы

[1] Ivankin E., Bazenkov N. Audio-Visual Stuttering Detection in Russian Video Recordings. Moscow Institute of Physics and Technology, 2026.

[2] Lea C., Mitra V., Joshi A., Kajarekar S., Bigham J. P. SEP-28k: A Dataset for Stuttering Event Detection from Podcasts with People Who Stutter. ICASSP, 2021, pp. 6798–6802.

[3] Baevski A., Zhou H., Mohamed A., Auli M. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. Advances in Neural Information Processing Systems (NeurIPS), 2020, vol. 33, pp. 12449–12460.

[4] Chen S., Wang C., Chen Z. et al. WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing. IEEE Journal of Selected Topics in Signal Processing, 2022, vol. 16, no. 6, pp. 1505–1518.

[5] Radford A., Kim J. W., Xu T. et al. Robust Speech Recognition via Large-Scale Weak Supervision. International Conference on Machine Learning (ICML), 2023, pp. 28492–28518.

[6] Lugaresi C., Tang J., Nash H. et al. MediaPipe: A Framework for Building Perception Pipelines. CVPR Workshop, 2019.

[7] Bai S., Kolter J. Z., Koltun V. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv preprint arXiv:1803.01271, 2018.

[8] Lin T. Y., Goyal P., Girshick R., He K., Dollar P. Focal Loss for Dense Object Detection. IEEE International Conference on Computer Vision (ICCV), 2017, pp. 2980–2988.

Приложение А. Полные листинги программного кода

В данном приложении приведены полные исходные тексты всех разработанных программных модулей исследовательского конвейера.

А.1. Скрипт загрузки и стандартизации датасета (sep28k_data_preparation.py)

```
import os
import io
import pandas as pd
import soundfile as sf
import librosa
from datasets import load_dataset, Audio
from pathlib import Path
from dotenv import load_dotenv

def download_and_prepare_sep28k():
    script_dir = Path(__file__).resolve().parent
    base_dir = script_dir.parent

    cache_dir = base_dir / "hf_cache"
    output_dir = base_dir / "data" / "sep28k_audio"

    cache_dir.mkdir(parents=True, exist_ok=True)
    output_dir.mkdir(parents=True, exist_ok=True)

    load_dotenv()

    hf_token = os.getenv('HF_TOKEN')

    if not hf_token:
        raise ValueError('Token error')

    print(f"Project root folder: {base_dir}")
    print(f"Audio files will be saved to: {output_dir}")
    print("Starting SEP-28k dataset download...")

    dataset = load_dataset(
        "saeedzou/sep28k-fluencybank-stutter-dataset",
        split="train",
        cache_dir=str(cache_dir),
        token=hf_token
    )

    dataset = dataset.cast_column("audio", Audio(decode=False))

    metadata = []

    for i, item in enumerate(dataset):
        audio_bytes = item['audio']['bytes']

        audio_array, orig_sr = sf.read(io.BytesIO(audio_bytes))

        if len(audio_array.shape) > 1:
```

```
            audio_array = audio_array.mean(axis=1)

        if orig_sr != 16000:
            audio_array = librosa.resample(y=audio_array, orig_sr=orig_sr,
    target_sr=16000)

        file_name = f"clip_{i:05d}.wav"
        file_path = output_dir / file_name

        sf.write(str(file_path), audio_array, 16000)

        row = {
            'file_name': file_name,
            'prolongation': item.get('Prolongation', 0),
            'block': item.get('Block', 0),
            'sound_repetition': item.get('SoundRep', 0),
            'word_repetition': item.get('WordRep', 0),
            'interjection': item.get('Interjection', 0),
            'is_stutter': 1 if any([
                item.get('Prolongation', 0) > 0,
                item.get('Block', 0) > 0,
                item.get('SoundRep', 0) > 0,
                item.get('WordRep', 0) > 0
            ]) else 0
        }
        metadata.append(row)

        if (i + 1) % 1000 == 0:
            print(f"Processed {i + 1} files...")

    df = pd.DataFrame(metadata)
    df.to_csv(output_dir / "metadata.csv", index=False)

    print(f"Saved {len(df)} files. Metadata written to {output_dir / 'metadata.csv'}")
    print(f"\nYou can delete the cache folder: {cache_dir}")
    print(f"Only the following folder will be used for training: {output_dir}")

if __name__ == "__main__":
    download_and_prepare_sep28k()
```

A.2. Скрипт разведочного анализа данных (sep28k_data_analysis.py)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

def analyze_dataset():
    script_dir = Path(__file__).resolve().parent
    base_dir = script_dir.parent
    csv_path = base_dir / "data" / "sep28k_audio" / "metadata.csv"

    if not csv_path.exists():
        print(f"File {csv_path} not found!")
        return
```

```
    df = pd.read_csv(csv_path)

    print(f"Total audio fragments: {len(df)}")

    class_counts = df['is_stutter'].value_counts()
    class_percentages = df['is_stutter'].value_counts(normalize=True) * 100

    print("\n--- Binary class balance ---")
    print(f"Fluent speech (0): {class_counts.get(0, 0)} ({class_percentages.get(0, 0):.2f}%)")
    print(f"Stutter (1):     {class_counts.get(1, 0)} ({class_percentages.get(1, 0):.2f}%)")

    stutter_types = ['prolongation', 'block', 'sound_repetition', 'word_repetition', 'interjection']
    print("\n--- Occurrence of specific types (can overlap) ---")

    type_stats = {}
    for col in stutter_types:
        count = (df[col] > 0).sum()
        percentage = (count / len(df)) * 100
        type_stats[col] = count
        print(f"{col.ljust(20)}: {count} ({percentage:.2f}%)")

    sns.set_theme(style="whitegrid")
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    sns.countplot(data=df, x='is_stutter', ax=axes[0], palette=['#2ecc71', '#e74c3c'])
    axes[0].set_title('Binary Class Balance (0 = Fluent, 1 = Stutter)', fontsize=14)
    axes[0].set_xlabel('Class')
    axes[0].set_ylabel('Number of fragments')

    types_s = pd.Series(type_stats).sort_values(ascending=False)
    sns.barplot(x=types_s.values, y=types_s.index, ax=axes[1], palette='viridis')
    axes[1].set_title('Distribution of Disfluency Types', fontsize=14)
    axes[1].set_xlabel('Number of fragments')

    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    analyze_dataset()
```

A.3. Извлечение агрегированных признаков (extract_features.py)

```
import os
import torch
import pandas as pd
import numpy as np
import soundfile as sf
from transformers import Wav2Vec2FeatureExtractor, Wav2Vec2Model
from pathlib import Path
from tqdm import tqdm

```

```python
def extract_embeddings():
    script_dir = Path(__file__).resolve().parent
    base_dir = script_dir.parent
    audio_dir = base_dir / "data" / "sep28k_audio"
    csv_path = audio_dir / "metadata.csv"
    output_dir = base_dir / "data" / "sep28k_embeddings"

    output_dir.mkdir(parents=True, exist_ok=True)

    if not csv_path.exists():
        print(f"File {csv_path} not found!")
        return

    df = pd.read_csv(csv_path)

    print("Loading Wav2Vec2-XLSR-53 model...")
    model_name = "facebook/wav2vec2-large-xlsr-53"

    feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(model_name)
    model = Wav2Vec2Model.from_pretrained(model_name)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)
    model.eval()

    print(f"Model loaded on device: {device}")
    print(f"Starting feature extraction for {len(df)} files...")

    with torch.no_grad():
        for index, row in tqdm(df.iterrows(), total=len(df)):
            file_name = row['file_name']
            file_path = audio_dir / file_name

            output_file = output_dir / file_name.replace(".wav", ".npy")
            if output_file.exists():
                continue

            if not file_path.exists():
                continue

            speech_array, sr = sf.read(str(file_path))

            inputs = feature_extractor(
                speech_array,
                sampling_rate=16000,
                return_tensors="pt",
                padding=True
            )

            inputs = {key: val.to(device) for key, val in inputs.items()}

            outputs = model(**inputs)

            hidden_states = outputs.last_hidden_state
            embedding = hidden_states.mean(dim=1).squeeze().cpu().numpy()

            np.save(str(output_file), embedding)

    print(f"\nAll embeddings saved to: {output_dir}")

```

```python
if __name__ == "__main__":
    extract_embeddings()
```

A.4. Пофреймовая экстракция Wav2Vec2 (extract_features_seq.py)

```python
"""
Step 1: Extract FRAME-LEVEL embeddings from wav2vec2-large-xlsr-53.
Instead of mean pooling, we save the full T x 1024 matrix, where T is the number of frames.
This allows the TCN classifier to detect local temporal anomalies.

Output: data/sep28k_seq_embeddings/<clip_name>.npy - array shape (T, 1024)
"""

import os
import torch
import pandas as pd
import numpy as np
import soundfile as sf
from transformers import Wav2Vec2FeatureExtractor, Wav2Vec2Model
from pathlib import Path
from tqdm import tqdm


def extract_sequential_embeddings():
    script_dir = Path(__file__).resolve().parent
    base_dir = script_dir.parent
    audio_dir = base_dir / "data" / "sep28k_audio"
    csv_path = audio_dir / "metadata.csv"
    output_dir = base_dir / "data" / "sep28k_seq_embeddings"

    output_dir.mkdir(parents=True, exist_ok=True)

    if not csv_path.exists():
        print(f"[ERROR] File {csv_path} not found!")
        return

    df = pd.read_csv(csv_path)

    already_done = sum(
        1 for _, row in df.iterrows()
        if (output_dir / row['file_name'].replace(".wav", ".npy")).exists()
    )
    remaining = len(df) - already_done
    print(f"[INFO] Total files: {len(df)}, already processed: {already_done}, remaining: {remaining}")

    if remaining == 0:
        print("[OK] All files have been processed!")
        return

    print("[LOAD] Loading Wav2Vec2-XLSR-53...")
    model_name = "facebook/wav2vec2-large-xlsr-53"
    feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(model_name)
    model = Wav2Vec2Model.from_pretrained(model_name)
```

```
49
50      device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
51      model.to(device)
52      model.eval()
53
54      for param in model.parameters():
55          param.requires_grad = False
56
57      print(f"[DEVICE] {device}")
58      print(f"[START] Starting sequential extraction for {remaining} files...\n")
59
60      processed = 0
61      errors = 0
62
63      with torch.no_grad():
64          pbar = tqdm(df.iterrows(), total=len(df), desc="Extraction", colour='green')
65          for _, row in pbar:
66              file_name = row['file_name']
67              output_file = output_dir / file_name.replace(".wav", ".npy")
68
69              if output_file.exists():
70                  continue
71
72              file_path = audio_dir / file_name
73              if not file_path.exists():
74                  errors += 1
75                  continue
76
77              try:
78                  speech_array, sr = sf.read(str(file_path))
79
80                  inputs = feature_extractor(
81                      speech_array,
82                      sampling_rate=16000,
83                      return_tensors="pt",
84                      padding=True
85                  )
86                  inputs = {key: val.to(device) for key, val in inputs.items()}
87
88                  outputs = model(**inputs)
89                  hidden_states = outputs.last_hidden_state
90
91                  seq_embedding = hidden_states.squeeze(0).cpu().numpy()
92                  np.save(str(output_file), seq_embedding)
93
94                  processed += 1
95
96              except Exception as e:
97                  errors += 1
98                  tqdm.write(f"[WARN] Error processing {file_name}: {e}")
99
100             pbar.set_postfix(done=processed, err=errors)
101
102     print(f"\n{'='*50}")
103     print(f"[DONE] Extraction complete!")
104     print(f"   Processed: {processed}")
105     print(f"   Errors: {errors}")
106     print(f"   Saved to: {output_dir}")
107     print(f"{'='*50}")
108
```

```
109
110 if __name__ == "__main__":
111     extract_sequential_embeddings()
```

A.5. Пофреймовая экстракция WavLM (extract_features_wavlm.py)

```
1  """
2  Step 2: Extract frame-level embeddings from WavLM-Base-Plus.
3  WavLM was trained considering paralinguistics and noise, providing
4  robust features for disfluency detection.
5
6  Output: data/sep28k_seq_wavlm/<clip_name>.npy - array shape (T, 768)
7  """
8
9  import torch
10 import pandas as pd
11 import numpy as np
12 import soundfile as sf
13 from transformers import WavLMModel, Wav2Vec2FeatureExtractor
14 from pathlib import Path
15 from tqdm import tqdm
16
17
18 def extract_wavlm_embeddings():
19     script_dir = Path(__file__).resolve().parent
20     base_dir = script_dir.parent
21     audio_dir = base_dir / "data" / "sep28k_audio"
22     csv_path = audio_dir / "metadata.csv"
23     output_dir = base_dir / "data" / "sep28k_seq_wavlm"
24
25     output_dir.mkdir(parents=True, exist_ok=True)
26
27     if not csv_path.exists():
28         print(f"[ERROR] File {csv_path} not found!")
29         return
30
31     df = pd.read_csv(csv_path)
32
33     already_done = sum(
34         1 for _, row in df.iterrows()
35         if (output_dir / row['file_name'].replace(".wav", ".npy")).exists()
36     )
37     remaining = len(df) - already_done
38     print(f"[INFO] Total files: {len(df)}, already processed: {already_done}, remaining:
       {remaining}")
39
40     if remaining == 0:
41         print("[OK] All files have been processed!")
42         return
43
44     print("[LOAD] Loading WavLM-Base-Plus...")
45     model_name = "microsoft/wavlm-base-plus"
46
47     feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(model_name)
48     model = WavLMModel.from_pretrained(model_name)
```

```
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)
    model.eval()

    for param in model.parameters():
        param.requires_grad = False

    print(f"[DEVICE] {device}")
    print(f"[START] Starting WavLM sequential extraction for {remaining} files...\n")

    processed = 0
    errors = 0

    with torch.no_grad():
        pbar = tqdm(df.iterrows(), total=len(df), desc="WavLM Extraction", colour='blue')
        for _, row in pbar:
            file_name = row['file_name']
            output_file = output_dir / file_name.replace(".wav", ".npy")

            if output_file.exists():
                continue

            file_path = audio_dir / file_name
            if not file_path.exists():
                errors += 1
                continue

            try:
                speech_array, sr = sf.read(str(file_path))

                inputs = feature_extractor(
                    speech_array,
                    sampling_rate=16000,
                    return_tensors="pt",
                    padding=True
                )
                inputs = {key: val.to(device) for key, val in inputs.items()}

                outputs = model(**inputs)
                hidden_states = outputs.last_hidden_state

                seq_embedding = hidden_states.squeeze(0).cpu().numpy()
                np.save(str(output_file), seq_embedding)

                processed += 1

            except Exception as e:
                errors += 1
                tqdm.write(f"[WARN] Error processing {file_name}: {e}")

            pbar.set_postfix(done=processed, err=errors)

    print(f"\n{'='*50}")
    print(f"[DONE] WavLM extraction complete!")
    print(f"   Processed: {processed}")
    print(f"   Errors: {errors}")
    print(f"   Saved to: {output_dir}")
    print(f"{'='*50}")
```

```
109
110 if __name__ == "__main__":
111     extract_wavlm_embeddings()
```

A.6. Пофреймовая экстракция Whisper (extract_features_whisper.py)

```
1  """
2  Step 2: Extract frame-level embeddings from Whisper encoder (small).
3  Whisper is a multilingual model trained on 680k hours of audio.
4  The encoder captures prosodic anomalies and rhythmic disruptions.
5
6  Output: data/sep28k_seq_whisper/<clip_name>.npy - array shape (T, 768)
7  """
8
9  import torch
10 import pandas as pd
11 import numpy as np
12 import soundfile as sf
13 from transformers import WhisperModel, WhisperFeatureExtractor
14 from pathlib import Path
15 from tqdm import tqdm
16
17
18 def extract_whisper_embeddings():
19     script_dir = Path(__file__).resolve().parent
20     base_dir = script_dir.parent
21     audio_dir = base_dir / "data" / "sep28k_audio"
22     csv_path = audio_dir / "metadata.csv"
23     output_dir = base_dir / "data" / "sep28k_seq_whisper"
24
25     output_dir.mkdir(parents=True, exist_ok=True)
26
27     if not csv_path.exists():
28         print(f"[ERROR] File {csv_path} not found!")
29         return
30
31     df = pd.read_csv(csv_path)
32
33     already_done = sum(
34         1 for _, row in df.iterrows()
35         if (output_dir / row['file_name'].replace(".wav", ".npy")).exists()
36     )
37     remaining = len(df) - already_done
38     print(f"[INFO] Total files: {len(df)}, already processed: {already_done}, remaining:
       {remaining}")
39
40     if remaining == 0:
41         print("[OK] All files have been processed!")
42         return
43
44     print("[LOAD] Loading Whisper-small encoder...")
45     model_name = "openai/whisper-small"
46
47     feature_extractor = WhisperFeatureExtractor.from_pretrained(model_name)
48     model = WhisperModel.from_pretrained(model_name)
```

```
49
50      device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
51      model.to(device)
52      model.eval()
53
54      for param in model.parameters():
55          param.requires_grad = False
56
57      print(f"[DEVICE] {device}")
58      print(f"[START] Starting Whisper sequential extraction for {remaining} files...\n")
59
60      processed = 0
61      errors = 0
62
63      with torch.no_grad():
64          pbar = tqdm(df.iterrows(), total=len(df), desc="Whisper Extraction",
    colour='yellow')
65          for _, row in pbar:
66              file_name = row['file_name']
67              output_file = output_dir / file_name.replace(".wav", ".npy")
68
69              if output_file.exists():
70                  continue
71
72              file_path = audio_dir / file_name
73              if not file_path.exists():
74                  errors += 1
75                  continue
76
77              try:
78                  speech_array, sr = sf.read(str(file_path))
79
80                  inputs = feature_extractor(
81                      speech_array,
82                      sampling_rate=16000,
83                      return_tensors="pt",
84                      padding="max_length",
85                      max_length=16000 * 30,
86                  )
87                  input_features = inputs.input_features.to(device)
88
89                  encoder_outputs = model.encoder(input_features)
90                  hidden_states = encoder_outputs.last_hidden_state
91
92                  real_duration_s = len(speech_array) / 16000
93                  real_frames = int(np.ceil(real_duration_s * 16000 / 320))
94                  real_frames = min(real_frames, hidden_states.shape[1])
95
96                  seq_embedding = hidden_states[0, :real_frames, :].cpu().numpy()
97                  np.save(str(output_file), seq_embedding)
98
99                  processed += 1
100
101             except Exception as e:
102                 errors += 1
103                 tqdm.write(f"[WARN] Error processing {file_name}: {e}")
104
105             pbar.set_postfix(done=processed, err=errors)
106
107     print(f"\n{'='*50}")
```

```
    print(f"[DONE] Whisper extraction complete!")
    print(f"   Processed: {processed}")
    print(f"   Errors: {errors}")
    print(f"   Saved to: {output_dir}")
    print(f"{'='*50}")


if __name__ == "__main__":
    extract_whisper_embeddings()
```

A.7. Общие утилиты, функции потерь и оптимизация ОЗУ (utils.py)

```
"""
Common utilities for the multimodal stuttering detection project.
Contains: metrics, datasets, loss functions, constants.
"""

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset
import numpy as np
import pandas as pd
from pathlib import Path
from tqdm import tqdm
from sklearn.metrics import (
    f1_score, precision_score, recall_score,
    roc_auc_score, average_precision_score,
    confusion_matrix
)

MAX_SEQ_LEN = 150
SAMPLE_RATE = 16000

ENCODER_DIMS = {
    "wav2vec2": 1024,
    "wavlm": 768,
    "whisper": 768,
}

def get_project_root():
    return Path(__file__).resolve().parent.parent


class FocalLoss(nn.Module):
    def __init__(self, alpha=0.25, gamma=2.0):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma

    def forward(self, logits, targets):
        bce = F.binary_cross_entropy_with_logits(logits, targets, reduction='none')
        pt = torch.exp(-bce)
        focal = self.alpha * (1 - pt) ** self.gamma * bce
        return focal.mean()
```

```python


class MeanPooledDataset(Dataset):
    def __init__(self, df, embeddings_dir, desc="Loading"):
        self.data = []
        self.labels = []
        df = df.reset_index(drop=True)
        for idx in tqdm(range(len(df)), desc=f"RAM [{desc}]", leave=False):
            fname = df.loc[idx, 'file_name'].replace(".wav", ".npy")
            emb = np.load(str(embeddings_dir / fname))
            self.data.append(emb.astype(np.float16))
            self.labels.append(df.loc[idx, 'is_stutter'])

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return (
            torch.tensor(self.data[idx], dtype=torch.float32),
            torch.tensor(self.labels[idx], dtype=torch.float32),
        )


class SequentialDataset(Dataset):
    def __init__(self, df, embeddings_dir, max_seq_len=MAX_SEQ_LEN, desc="Loading"):
        self.data = []
        self.labels = []
        self.masks = []
        self.max_seq_len = max_seq_len
        df = df.reset_index(drop=True)

        skipped = 0
        for idx in tqdm(range(len(df)), desc=f"RAM [{desc}]", leave=False):
            fname = df.loc[idx, 'file_name'].replace(".wav", ".npy")
            path = embeddings_dir / fname
            if not path.exists():
                skipped += 1
                continue

            emb = np.load(str(path))
            T, D = emb.shape

            if T >= max_seq_len:
                emb_padded = emb[:max_seq_len]
                mask = np.ones(max_seq_len, dtype=np.float32)
            else:
                pad_len = max_seq_len - T
                emb_padded = np.pad(emb, ((0, pad_len), (0, 0)), mode='constant')
                mask = np.concatenate([np.ones(T, dtype=np.float32),
                                       np.zeros(pad_len, dtype=np.float32)])

            self.data.append(emb_padded.astype(np.float16))
            self.masks.append(mask)
            self.labels.append(df.loc[idx, 'is_stutter'])

        if skipped > 0:
            print(f"  [WARN] Skipped {skipped} files (missing .npy)")

    def __len__(self):
        return len(self.data)
```

```
104
105     def __getitem__(self, idx):
106         return (
107             torch.tensor(self.data[idx], dtype=torch.float32),
108             torch.tensor(self.masks[idx], dtype=torch.float32),
109             torch.tensor(self.labels[idx], dtype=torch.float32),
110         )
111
112
113 def compute_metrics(y_true, y_pred, y_prob=None):
114     y_true = np.asarray(y_true)
115     y_pred = np.asarray(y_pred)
116
117     tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
118     specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0
119
120     metrics = {
121         'f1': f1_score(y_true, y_pred, zero_division=0),
122         'precision': precision_score(y_true, y_pred, zero_division=0),
123         'recall': recall_score(y_true, y_pred, zero_division=0),
124         'specificity': specificity,
125     }
126
127     if y_prob is not None:
128         y_prob = np.asarray(y_prob)
129         try:
130             metrics['roc_auc'] = roc_auc_score(y_true, y_prob)
131         except ValueError:
132             metrics['roc_auc'] = 0.0
133         try:
134             metrics['ap'] = average_precision_score(y_true, y_prob)
135         except ValueError:
136             metrics['ap'] = 0.0
137
138     return metrics
139
140
141 def print_metrics(metrics, prefix=""):
142     parts = []
143     for key in ['f1', 'precision', 'recall', 'specificity', 'roc_auc', 'ap']:
144         if key in metrics:
145             parts.append(f"{key.upper()}: {metrics[key]:.4f}")
146     print(f"{prefix}{' | '.join(parts)}")
147
148
149 def metrics_to_row(metrics, method_name, **extra):
150     row = {'method': method_name}
151     row.update(metrics)
152     row.update(extra)
153     return row
```

A.8. Обучение базового классификатора MLP (train_mlp.py)

```
1 import os
2 import time
3 import torch
4 import torch.nn as nn
```

```
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import pandas as pd
import numpy as np
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, f1_score, precision_score,
    recall_score, roc_auc_score
from tqdm import tqdm

class FocalLoss(nn.Module):
    def __init__(self, alpha=1, gamma=2):
        super(FocalLoss, self).__init__()
        self.alpha = alpha
        self.gamma = gamma

    def forward(self, inputs, targets):
        BCE_loss = F.binary_cross_entropy_with_logits(inputs, targets, reduction='none')
        pt = torch.exp(-BCE_loss)
        F_loss = self.alpha * (1 - pt)**self.gamma * BCE_loss
        return torch.mean(F_loss)

class StutterDataset(Dataset):
    def __init__(self, df, embeddings_dir, desc="Loading"):
        self.df = df.reset_index(drop=True)
        self.embeddings_dir = embeddings_dir

        self.data = []
        self.labels = []

        for idx in tqdm(range(len(self.df)), desc=f"To RAM [{desc}]", leave=False):
            file_name = self.df.loc[idx, 'file_name']
            npy_name = file_name.replace(".wav", ".npy")
            embedding_path = self.embeddings_dir / npy_name

            embedding = np.load(str(embedding_path))
            label = self.df.loc[idx, 'is_stutter']

            self.data.append(embedding.astype(np.float16))
            self.labels.append(label)

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        return torch.tensor(self.data[idx], dtype=torch.float32),
    torch.tensor(self.labels[idx], dtype=torch.float32)

class AudioClassifier(nn.Module):
    def __init__(self, input_dim=1024):
        super(AudioClassifier, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(512, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
```

```
            nn.Dropout(0.3),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.network(x).squeeze(1)

def train_model():
    script_dir = Path(__file__).resolve().parent
    base_dir = script_dir.parent
    embeddings_dir = base_dir / "data" / "sep28k_embeddings"
    csv_path = base_dir / "data" / "sep28k_audio" / "metadata.csv"

    print("\n" + "="*60)
    print("[INFO] Initializing data and loading metadata...")
    print("="*60)

    df = pd.read_csv(csv_path)

    tqdm.pandas(desc="Checking .npy files", leave=False)
    df['npy_exists'] = df['file_name'].progress_apply(lambda x: (embeddings_dir / x.replace('.wav', '.npy')).exists())
    df = df[df['npy_exists'] == True]

    train_df, val_df = train_test_split(df, test_size=0.2, random_state=42, stratify=df['is_stutter'])

    train_dataset = StutterDataset(train_df, embeddings_dir, desc="Train")
    val_dataset = StutterDataset(val_df, embeddings_dir, desc="Valid")

    train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"\n[DEVICE] Compute device: {device.type.upper()}")

    model = AudioClassifier().to(device)

    criterion = FocalLoss(alpha=0.25, gamma=2.0)
    optimizer = optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-3)

    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.5, patience=2)

    epochs = 40
    best_roc_auc = 0.0

    print("\n" + "="*60)
    print("[START] Starting MLP classifier training loop...")
    print("="*60 + "\n")

    for epoch in range(epochs):
        epoch_start_time = time.time()

        model.train()
        train_loss = 0.0

        train_loop = tqdm(train_loader, desc=f"Epoch {epoch+1:02d}/{epochs} [Train]", leave=False)

```

```
        for embeddings, labels in train_loop:
            embeddings, labels = embeddings.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(embeddings)
            loss = criterion(outputs, labels)

            loss.backward()
            optimizer.step()
            train_loss += loss.item()

            train_loop.set_postfix(loss=f"{loss.item():.4f}")

        model.eval()
        val_preds = []
        val_probs = []
        val_true = []

        val_loop = tqdm(val_loader, desc=f"Epoch {epoch+1:02d}/{epochs} [Valid]",
    leave=False)

        with torch.no_grad():
            for embeddings, labels in val_loop:
                embeddings, labels = embeddings.to(device), labels.to(device)
                outputs = model(embeddings)

                probs = torch.sigmoid(outputs)
                preds = (probs > 0.5).float()

                val_probs.extend(probs.cpu().numpy())
                val_preds.extend(preds.cpu().numpy())
                val_true.extend(labels.cpu().numpy())

        val_f1 = f1_score(val_true, val_preds, zero_division=0)
        val_prec = precision_score(val_true, val_preds, zero_division=0)
        val_rec = recall_score(val_true, val_preds, zero_division=0)
        val_roc = roc_auc_score(val_true, val_probs)

        avg_train_loss = train_loss / len(train_loader)

        scheduler.step(val_roc)

        epoch_duration = time.time() - epoch_start_time
        mins, secs = divmod(epoch_duration, 60)

        print(f"Epoch {epoch+1:02d}/{epochs} [{int(mins):02d}:{int(secs):02d}] | Loss:
    {avg_train_loss:.4f} | Val ROC-AUC: {val_roc:.4f} | F1: {val_f1:.4f} | Prec:
    {val_prec:.4f} | Rec: {val_rec:.4f}")

        if val_roc > best_roc_auc:
            best_roc_auc = val_roc
            torch.save(model.state_dict(), base_dir / "scripts" / "best_mlp_model.pth")
            print(f"  [SAVE] New best model saved! (ROC-AUC: {best_roc_auc:.4f})")

    print("\n" + "="*60)
    print(f"[DONE] Training complete. Best validation ROC-AUC: {best_roc_auc:.4f}")
    print("="*60)

if __name__ == "__main__":
    train_model()
```

A.9. Архитектура и обучение временной сверточной сети (train_tcn.py)

```
"""
Step 1: TCN (Temporal Convolutional Network) classifier for stuttering.
Works on sequential embeddings T x 1024 from wav2vec2.

Architecture: 4 residual dilated 1D conv blocks (dilation 1,2,4,8)
-> Global Average Pooling -> Linear -> sigmoid.
"""

import os
import time
import argparse
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import DataLoader
import numpy as np
import pandas as pd
from pathlib import Path
from sklearn.model_selection import train_test_split
from tqdm import tqdm

from utils import (
    FocalLoss, SequentialDataset, MAX_SEQ_LEN,
    compute_metrics, print_metrics, get_project_root
)


class ResidualBlock(nn.Module):
    def __init__(self, channels, kernel_size=3, dilation=1, dropout=0.2):
        super().__init__()
        padding = (kernel_size - 1) * dilation // 2

        self.conv1 = nn.Conv1d(channels, channels, kernel_size,
                               padding=padding, dilation=dilation)
        self.bn1 = nn.BatchNorm1d(channels)
        self.conv2 = nn.Conv1d(channels, channels, kernel_size=1)
        self.bn2 = nn.BatchNorm1d(channels)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        residual = x
        out = self.dropout(F.relu(self.bn1(self.conv1(x))))
        out = self.bn2(self.conv2(out))
        if out.shape[-1] != residual.shape[-1]:
            out = out[..., :residual.shape[-1]]
        return F.relu(out + residual)


class StutterTCN(nn.Module):
    def __init__(self, input_dim=1024, hidden_dim=256, num_blocks=4,
                 kernel_size=5, dropout=0.3):
        super().__init__()

        self.input_proj = nn.Sequential(
```

```
            nn.Conv1d(input_dim, hidden_dim, kernel_size=1),
            nn.BatchNorm1d(hidden_dim),
            nn.ReLU(),
        )

        dilations = [2 ** i for i in range(num_blocks)]
        self.blocks = nn.ModuleList([
            ResidualBlock(hidden_dim, kernel_size, dilation=d, dropout=dropout)
            for d in dilations
        ])

        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim // 2, 1),
        )

    def forward(self, x, mask=None):
        x = x.permute(0, 2, 1)
        x = self.input_proj(x)

        for block in self.blocks:
            x = block(x)

        x = x.permute(0, 2, 1)
        if mask is not None:
            mask_expanded = mask.unsqueeze(-1)
            x = (x * mask_expanded).sum(dim=1) / mask_expanded.sum(dim=1).clamp(min=1)
        else:
            x = x.mean(dim=1)

        logits = self.classifier(x).squeeze(-1)
        return logits


def train_tcn(args):
    base_dir = get_project_root()
    embeddings_dir = base_dir / "data" / args.embeddings_subdir
    csv_path = base_dir / "data" / "sep28k_audio" / "metadata.csv"
    save_path = base_dir / "scripts" / f"best_tcn_{args.encoder_name}.pth"

    print("\n" + "=" * 60)
    print(f"[INFO] TCN Training | Encoder: {args.encoder_name}")
    print(f"[INFO] Embeddings: {embeddings_dir}")
    print(f"[INFO] Input dim: {args.input_dim}")
    print("=" * 60)

    df = pd.read_csv(csv_path)

    df['npy_exists'] = df['file_name'].apply(
        lambda x: (embeddings_dir / x.replace('.wav', '.npy')).exists()
    )
    df = df[df['npy_exists']].drop(columns=['npy_exists'])
    print(f"[DATA] Valid files with embeddings: {len(df)}")

    train_df, val_df = train_test_split(
        df, test_size=0.2, random_state=42, stratify=df['is_stutter']
    )
    print(f"[DATA] Train: {len(train_df)} | Val: {len(val_df)}")
```

```
116
117     train_dataset = SequentialDataset(train_df, embeddings_dir,
118                                       max_seq_len=args.max_seq_len, desc="Train")
119     val_dataset = SequentialDataset(val_df, embeddings_dir,
120                                     max_seq_len=args.max_seq_len, desc="Valid")
121
122     train_loader = DataLoader(train_dataset, batch_size=args.batch_size,
123                               shuffle=True, num_workers=0, pin_memory=True)
124     val_loader = DataLoader(val_dataset, batch_size=args.batch_size,
125                             shuffle=False, num_workers=0, pin_memory=True)
126
127     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
128     print(f"\n[DEVICE] Compute device: {device}")
129
130     model = StutterTCN(
131         input_dim=args.input_dim,
132         hidden_dim=args.hidden_dim,
133         num_blocks=args.num_blocks,
134         kernel_size=args.kernel_size,
135         dropout=args.dropout,
136     ).to(device)
137
138     param_count = sum(p.numel() for p in model.parameters() if p.requires_grad)
139     print(f"[MODEL] Trainable parameters: {param_count:,}")
140
141     criterion = FocalLoss(alpha=0.25, gamma=2.0)
142     optimizer = optim.AdamW(model.parameters(), lr=args.lr,
        weight_decay=args.weight_decay)
143     scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=args.epochs,
        eta_min=1e-6)
144
145     best_roc_auc = 0.0
146     patience_counter = 0
147
148     print(f"\n{'='*60}")
149     print(f"[START] Training for {args.epochs} epochs")
150     print(f"{'='*60}\n")
151
152     for epoch in range(args.epochs):
153         epoch_start = time.time()
154
155         model.train()
156         train_loss = 0.0
157         train_steps = 0
158
159         pbar = tqdm(train_loader, desc=f"Epoch {epoch+1:02d}/{args.epochs} [Train]",
160                     leave=False, colour='cyan')
161         for embeddings, masks, labels in pbar:
162             embeddings = embeddings.to(device)
163             masks = masks.to(device)
164             labels = labels.to(device)
165
166             optimizer.zero_grad()
167             logits = model(embeddings, masks)
168             loss = criterion(logits, labels)
169             loss.backward()
170
171             torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
172
173             optimizer.step()
```

```
            train_loss += loss.item()
            train_steps += 1
            pbar.set_postfix(loss=f"{loss.item():.4f}")

        scheduler.step()

        model.eval()
        val_preds = []
        val_probs = []
        val_true = []

        with torch.no_grad():
            for embeddings, masks, labels in val_loader:
                embeddings = embeddings.to(device)
                masks = masks.to(device)

                logits = model(embeddings, masks)
                probs = torch.sigmoid(logits)
                preds = (probs > 0.5).float()

                val_probs.extend(probs.cpu().numpy())
                val_preds.extend(preds.cpu().numpy())
                val_true.extend(labels.numpy())

        metrics = compute_metrics(val_true, val_preds, val_probs)
        avg_loss = train_loss / max(train_steps, 1)
        duration = time.time() - epoch_start
        lr = optimizer.param_groups[0]['lr']

        print(f"Epoch {epoch+1:02d}/{args.epochs} [{duration:.1f}s] | "
              f"Loss: {avg_loss:.4f} | LR: {lr:.2e} | ", end="")
        print_metrics(metrics)

        current_roc = metrics.get('roc_auc', 0.0)

        if current_roc > best_roc_auc:
            best_roc_auc = current_roc
            patience_counter = 0
            torch.save({
                'epoch': epoch + 1,
                'model_state_dict': model.state_dict(),
                'optimizer_state_dict': optimizer.state_dict(),
                'best_roc_auc': best_roc_auc,
                'best_f1': metrics.get('f1', 0.0),
                'metrics': metrics,
                'args': vars(args),
            }, save_path)
            print(f"  [SAVE] New best model! ROC-AUC: {best_roc_auc:.4f} (F1:
    {metrics.get('f1', 0.0):.4f})")
        else:
            patience_counter += 1
            if patience_counter >= args.patience:
                print(f"\n[STOP] Early stopping: {args.patience} epochs without ROC-AUC
    improvement.")
                break

    print(f"\n{'='*60}")
    print(f"[DONE] Training complete. Best val ROC-AUC: {best_roc_auc:.4f}")
    print(f"[INFO] Model saved to: {save_path}")
    print(f"{'='*60}")
```

```
232
233     return best_roc_auc
234
235
236 def parse_args():
237     parser = argparse.ArgumentParser(description="Train TCN classifier for stuttering
    detection")
238     parser.add_argument('--encoder_name', type=str, default='wav2vec2',
239                         choices=['wav2vec2', 'wavlm', 'whisper'])
240     parser.add_argument('--embeddings_subdir', type=str, default=None)
241     parser.add_argument('--max_seq_len', type=int, default=MAX_SEQ_LEN)
242     parser.add_argument('--input_dim', type=int, default=None)
243     parser.add_argument('--hidden_dim', type=int, default=256)
244     parser.add_argument('--num_blocks', type=int, default=4)
245     parser.add_argument('--kernel_size', type=int, default=5)
246     parser.add_argument('--dropout', type=float, default=0.3)
247     parser.add_argument('--batch_size', type=int, default=32)
248     parser.add_argument('--epochs', type=int, default=50)
249     parser.add_argument('--lr', type=float, default=3e-4)
250     parser.add_argument('--weight_decay', type=float, default=1e-3)
251     parser.add_argument('--patience', type=int, default=10)
252
253     args = parser.parse_args()
254
255     from utils import ENCODER_DIMS
256     if args.input_dim is None:
257         args.input_dim = ENCODER_DIMS[args.encoder_name]
258
259     if args.embeddings_subdir is None:
260         subdir_map = {
261             'wav2vec2': 'sep28k_seq_embeddings',
262             'wavlm': 'sep28k_seq_wavlm',
263             'whisper': 'sep28k_seq_whisper',
264         }
265         args.embeddings_subdir = subdir_map[args.encoder_name]
266
267     return args
268
269 if __name__ == "__main__":
270     args = parse_args()
271     train_tcn(args)
```

A.10. Гибридная архитектура TCN + Multi-Head Attention (train_hybrid_tcn.py)

```
1 """
2 Step 1.5: Hybrid TCN + Multi-Head Attention classifier.
3 Combines local anomaly detection (TCN) with global context (Self-Attention).
4 """
5
6 import time
7 import argparse
8 import torch
9 import torch.nn as nn
10 import torch.optim as optim
11 import torch.nn.functional as F
```

```
from torch.utils.data import DataLoader
import numpy as np
import pandas as pd
from pathlib import Path
from sklearn.model_selection import train_test_split
from tqdm import tqdm

from utils import (
    FocalLoss, SequentialDataset, MAX_SEQ_LEN,
    compute_metrics, print_metrics, get_project_root
)

class ResidualBlock(nn.Module):
    def __init__(self, channels, kernel_size=3, dilation=1, dropout=0.2):
        super().__init__()
        padding = (kernel_size - 1) * dilation // 2
        self.conv1 = nn.Conv1d(channels, channels, kernel_size, padding=padding,
    dilation=dilation)
        self.bn1 = nn.BatchNorm1d(channels)
        self.conv2 = nn.Conv1d(channels, channels, kernel_size=1)
        self.bn2 = nn.BatchNorm1d(channels)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        residual = x
        out = self.dropout(F.relu(self.bn1(self.conv1(x))))
        out = self.bn2(self.conv2(out))
        if out.shape[-1] != residual.shape[-1]:
            out = out[..., :residual.shape[-1]]
        return F.relu(out + residual)


class StutterHybridTCN(nn.Module):
    def __init__(self, input_dim=768, hidden_dim=256, num_blocks=4, kernel_size=5,
    dropout=0.3):
        super().__init__()
        self.input_proj = nn.Sequential(
            nn.Conv1d(input_dim, hidden_dim, kernel_size=1),
            nn.BatchNorm1d(hidden_dim),
            nn.ReLU(),
        )
        dilations = [2 ** i for i in range(num_blocks)]
        self.blocks = nn.ModuleList([
            ResidualBlock(hidden_dim, kernel_size, dilation=d, dropout=dropout)
            for d in dilations
        ])

        self.attention = nn.MultiheadAttention(
            embed_dim=hidden_dim,
            num_heads=4,
            dropout=dropout,
            batch_first=True
        )
        self.layer_norm = nn.LayerNorm(hidden_dim)

        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim // 2, 1),
```

```
        )

    def forward(self, x, mask=None):
        x = x.permute(0, 2, 1)
        x = self.input_proj(x)
        for block in self.blocks:
            x = block(x)
        x = x.permute(0, 2, 1)

        if mask is not None:
            key_padding_mask = (mask == 0)
            attn_out, _ = self.attention(x, x, x, key_padding_mask=key_padding_mask)
        else:
            attn_out, _ = self.attention(x, x, x)

        x = self.layer_norm(x + attn_out)

        if mask is not None:
            mask_expanded = mask.unsqueeze(-1)
            x = (x * mask_expanded).sum(dim=1) / mask_expanded.sum(dim=1).clamp(min=1)
        else:
            x = x.mean(dim=1)

        return self.classifier(x).squeeze(-1)


def train_hybrid(args):
    base_dir = get_project_root()
    embeddings_dir = base_dir / "data" / args.embeddings_subdir
    csv_path = base_dir / "data" / "sep28k_audio" / "metadata.csv"
    save_path = base_dir / "scripts" / f"best_hybrid_{args.encoder_name}.pth"

    print(f"[INFO] Hybrid TCN+Attention Training | Encoder: {args.encoder_name}")
    print("=" * 60)

    df = pd.read_csv(csv_path)
    df['npy_exists'] = df['file_name'].apply(lambda x: (embeddings_dir /
    x.replace('.wav', '.npy')).exists())
    df = df[df['npy_exists']].drop(columns=['npy_exists'])

    train_df, val_df = train_test_split(df, test_size=0.2, random_state=42,
    stratify=df['is_stutter'])

    train_dataset = SequentialDataset(train_df, embeddings_dir,
    max_seq_len=args.max_seq_len, desc="Train")
    val_dataset = SequentialDataset(val_df, embeddings_dir,
    max_seq_len=args.max_seq_len, desc="Valid")

    train_loader = DataLoader(train_dataset, batch_size=args.batch_size, shuffle=True,
    num_workers=0, pin_memory=True)
    val_loader = DataLoader(val_dataset, batch_size=args.batch_size, shuffle=False,
    num_workers=0, pin_memory=True)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    model = StutterHybridTCN(
        input_dim=args.input_dim, hidden_dim=args.hidden_dim,
        num_blocks=args.num_blocks, kernel_size=args.kernel_size, dropout=args.dropout,
    ).to(device)

```

```
    criterion = FocalLoss(alpha=0.25, gamma=2.0)
    optimizer = optim.AdamW(model.parameters(), lr=args.lr,
    weight_decay=args.weight_decay)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=args.epochs,
    eta_min=1e-6)

    best_roc_auc = 0.0
    patience_counter = 0

    for epoch in range(args.epochs):
        epoch_start = time.time()
        model.train()
        train_loss = 0.0
        train_steps = 0

        pbar = tqdm(train_loader, desc=f"Epoch {epoch+1:02d}/{args.epochs} [Train]",
    leave=False)
        for embeddings, masks, labels in pbar:
            embeddings, masks, labels = embeddings.to(device), masks.to(device),
    labels.to(device)
            optimizer.zero_grad()
            logits = model(embeddings, masks)
            loss = criterion(logits, labels)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
            optimizer.step()
            train_loss += loss.item()
            train_steps += 1

        scheduler.step()
        model.eval()
        val_preds, val_probs, val_true = [], [], []

        with torch.no_grad():
            for embeddings, masks, labels in val_loader:
                embeddings, masks = embeddings.to(device), masks.to(device)
                logits = model(embeddings, masks)
                probs = torch.sigmoid(logits)
                val_probs.extend(probs.cpu().numpy())
                val_preds.extend((probs > 0.5).float().cpu().numpy())
                val_true.extend(labels.numpy())

        metrics = compute_metrics(val_true, val_preds, val_probs)
        avg_loss = train_loss / max(train_steps, 1)
        duration = time.time() - epoch_start

        print(f"Epoch {epoch+1:02d}/{args.epochs} [{duration:.1f}s] | Loss:
    {avg_loss:.4f} | ", end="")
        print_metrics(metrics)

        current_roc = metrics.get('roc_auc', 0.0)
        if current_roc > best_roc_auc:
            best_roc_auc = current_roc
            patience_counter = 0
            torch.save({
                'epoch': epoch + 1, 'model_state_dict': model.state_dict(),
                'best_roc_auc': best_roc_auc, 'metrics': metrics, 'args': vars(args),
            }, save_path)
            print(f"  [SAVE] New best Hybrid model! ROC-AUC: {best_roc_auc:.4f}")
        else:
```

```
            patience_counter += 1
            if patience_counter >= args.patience:
                print(f"\n[STOP] Early stopping: {args.patience} epochs without improvement.")
                break

    print(f"\n[DONE] Training complete. Best val ROC-AUC: {best_roc_auc:.4f}")

def parse_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--encoder_name', type=str, default='whisper', choices=['wav2vec2', 'wavlm', 'whisper'])
    parser.add_argument('--embeddings_subdir', type=str, default='sep28k_seq_whisper')
    parser.add_argument('--max_seq_len', type=int, default=MAX_SEQ_LEN)
    parser.add_argument('--input_dim', type=int, default=768)
    parser.add_argument('--hidden_dim', type=int, default=256)
    parser.add_argument('--num_blocks', type=int, default=4)
    parser.add_argument('--kernel_size', type=int, default=5)
    parser.add_argument('--dropout', type=float, default=0.3)
    parser.add_argument('--batch_size', type=int, default=32)
    parser.add_argument('--epochs', type=int, default=50)
    parser.add_argument('--lr', type=float, default=3e-4)
    parser.add_argument('--weight_decay', type=float, default=1e-3)
    parser.add_argument('--patience', type=int, default=10)
    return parser.parse_args()

if __name__ == "__main__":
    train_hybrid(parse_args())
```

A.11. Сравнение акустических энкодеров (compare_encoders.py)

```
"""
Step 2: Comparative metrics table across all encoders.
Loads saved checkpoints of the best TCN models for each encoder
and evaluates them on the validation set.

Run: python scripts/compare_encoders.py
"""

import torch
import pandas as pd
import numpy as np
from pathlib import Path
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader
from tabulate import tabulate

from utils import (
    SequentialDataset, MeanPooledDataset, compute_metrics, MAX_SEQ_LEN,
    get_project_root
)
from train_tcn import StutterTCN


ENCODER_CONFIGS = {
    'wav2vec2': {
        'embeddings_subdir': 'sep28k_seq_embeddings',
```

```
        'checkpoint': 'best_tcn_wav2vec2.pth',
        'input_dim': 1024,
    },
    'wavlm': {
        'embeddings_subdir': 'sep28k_seq_wavlm',
        'checkpoint': 'best_tcn_wavlm.pth',
        'input_dim': 768,
    },
    'whisper': {
        'embeddings_subdir': 'sep28k_seq_whisper',
        'checkpoint': 'best_tcn_whisper.pth',
        'input_dim': 768,
    },
}


def evaluate_tcn(encoder_name, config, base_dir, device, df_val):
    ckpt_path = base_dir / "scripts" / config['checkpoint']
    emb_dir = base_dir / "data" / config['embeddings_subdir']

    if not ckpt_path.exists():
        print(f"  [WARN] Checkpoint {ckpt_path.name} not found - skipping {encoder_name}")
        return None

    if not emb_dir.exists():
        print(f"  [WARN] Embeddings {emb_dir} not found - skipping {encoder_name}")
        return None

    checkpoint = torch.load(ckpt_path, map_location=device, weights_only=False)
    saved_args = checkpoint.get('args', {})

    model = StutterTCN(
        input_dim=config['input_dim'],
        hidden_dim=saved_args.get('hidden_dim', 256),
        num_blocks=saved_args.get('num_blocks', 4),
        kernel_size=saved_args.get('kernel_size', 5),
        dropout=saved_args.get('dropout', 0.3),
    ).to(device)
    model.load_state_dict(checkpoint['model_state_dict'])
    model.eval()

    val_dataset = SequentialDataset(
        df_val, emb_dir,
        max_seq_len=saved_args.get('max_seq_len', MAX_SEQ_LEN),
        desc=f"Val-{encoder_name}"
    )
    val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False, num_workers=0)

    all_probs, all_preds, all_true = [], [], []

    with torch.no_grad():
        for embeddings, masks, labels in val_loader:
            embeddings = embeddings.to(device)
            masks = masks.to(device)
            logits = model(embeddings, masks)
            probs = torch.sigmoid(logits)
            preds = (probs > 0.5).float()

            all_probs.extend(probs.cpu().numpy())
```

```
            all_preds.extend(preds.cpu().numpy())
            all_true.extend(labels.numpy())

    metrics = compute_metrics(all_true, all_preds, all_probs)
    metrics['encoder'] = f"TCN ({encoder_name})"
    return metrics


def evaluate_mlp(base_dir, device, df_val):
    from train_mlp import AudioClassifier

    ckpt_path = base_dir / "scripts" / "best_mlp_model.pth"
    emb_dir = base_dir / "data" / "sep28k_embeddings"

    if not ckpt_path.exists() or not emb_dir.exists():
        print("  [WARN] MLP checkpoint or embeddings not found - skipping")
        return None

    model = AudioClassifier(input_dim=1024).to(device)
    model.load_state_dict(torch.load(ckpt_path, map_location=device, weights_only=False))
    model.eval()

    val_dataset = MeanPooledDataset(df_val, emb_dir, desc="Val-MLP")
    val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False, num_workers=0)

    all_probs, all_preds, all_true = [], [], []

    with torch.no_grad():
        for embeddings, labels in val_loader:
            embeddings = embeddings.to(device)
            logits = model(embeddings)
            probs = torch.sigmoid(logits)
            preds = (probs > 0.5).float()

            all_probs.extend(probs.cpu().numpy())
            all_preds.extend(preds.cpu().numpy())
            all_true.extend(labels.numpy())

    metrics = compute_metrics(all_true, all_preds, all_probs)
    metrics['encoder'] = 'MLP (wav2vec2 mean-pooled)'
    return metrics


def main():
    base_dir = get_project_root()
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    csv_path = base_dir / "data" / "sep28k_audio" / "metadata.csv"
    df = pd.read_csv(csv_path)
    _, df_val = train_test_split(
        df, test_size=0.2, random_state=42, stratify=df['is_stutter']
    )

    print("\n" + "=" * 70)
    print("  [INFO] Encoder Comparison: Unified Validation Evaluation")
    print("=" * 70 + "\n")

    results = []

    print("[EVAL] MLP Baseline (mean-pooled wav2vec2)...")
```

```
    mlp_metrics = evaluate_mlp(base_dir, device, df_val)
    if mlp_metrics:
        results.append(mlp_metrics)

    for encoder_name, config in ENCODER_CONFIGS.items():
        print(f"[EVAL] TCN + {encoder_name}...")
        metrics = evaluate_tcn(encoder_name, config, base_dir, device, df_val)
        if metrics:
            results.append(metrics)

    if results:
        print("\n" + "=" * 70)
        print("  [RESULTS] COMPARATIVE TABLE")
        print("=" * 70 + "\n")

        df_results = pd.DataFrame(results)
        columns_order = ['encoder', 'roc_auc', 'ap', 'f1', 'precision', 'recall',
    'specificity']
        columns_present = [c for c in columns_order if c in df_results.columns]
        df_display = df_results[columns_present].copy()

        for col in columns_present:
            if col != 'encoder':
                df_display[col] = df_display[col].apply(lambda x: f"{x:.4f}")

        print(tabulate(df_display, headers='keys', tablefmt='grid', showindex=False))

        out_path = base_dir / "scripts" / "encoder_comparison.csv"
        df_results[columns_present].to_csv(out_path, index=False)
        print(f"\n[SAVE] Table saved to: {out_path}")
    else:
        print("[ERROR] No encoders were evaluated. Train the models first.")


if __name__ == "__main__":
    main()
```

A.12. Извлечение физической кинематики лица (extract_face_kinematics.py)

```
"""
Step 3: Extract facial kinematic features from video via MediaPipe Face Mesh.
Run: python scripts/extract_face_kinematics.py --video_dir <path> --output_dir <path>
"""

import argparse
import cv2
import numpy as np
import mediapipe as mp
from pathlib import Path
from tqdm import tqdm

UPPER_LIP_INNER = [82, 13, 312]
LOWER_LIP_INNER = [87, 14, 317]
LEFT_LIP_CORNER = 61
RIGHT_LIP_CORNER = 291
```

```
CHIN = 152
LEFT_JAW = 234
RIGHT_JAW = 454
LEFT_BROW_INNER = 70
RIGHT_BROW_INNER = 300
NOSE_TIP = 1
NOSE_BRIDGE = 6
LEFT_EYE_TOP = 159
LEFT_EYE_BOTTOM = 145
RIGHT_EYE_TOP = 386
RIGHT_EYE_BOTTOM = 374


def euclidean(p1, p2):
    return np.sqrt(np.sum((np.array(p1) - np.array(p2)) ** 2))


def get_landmark_coords(landmarks, idx, w=1.0, h=1.0):
    lm = landmarks[idx]
    return np.array([lm.x * w, lm.y * h, lm.z * w])


def compute_frame_features(landmarks, w, h):
    def lm(idx):
        return get_landmark_coords(landmarks, idx, w, h)

    face_scale = euclidean(lm(NOSE_BRIDGE), lm(NOSE_TIP))
    if face_scale < 1e-6:
        face_scale = 1.0

    upper_lip_pts = [lm(i) for i in UPPER_LIP_INNER]
    lower_lip_pts = [lm(i) for i in LOWER_LIP_INNER]
    vertical_dist = np.mean([euclidean(u, l) for u, l in zip(upper_lip_pts,
    lower_lip_pts)])
    horizontal_dist = euclidean(lm(LEFT_LIP_CORNER), lm(RIGHT_LIP_CORNER))
    mar = vertical_dist / max(horizontal_dist, 1e-6)

    lip_spread = horizontal_dist / face_scale
    jaw_disp = lm(CHIN)[1] / face_scale
    brow_dist = euclidean(lm(LEFT_BROW_INNER), lm(RIGHT_BROW_INNER)) / face_scale

    left_brow_h = (lm(LEFT_BROW_INNER)[1] - lm(LEFT_EYE_TOP)[1]) / face_scale
    right_brow_h = (lm(RIGHT_BROW_INNER)[1] - lm(RIGHT_EYE_TOP)[1]) / face_scale
    avg_brow_height = (left_brow_h + right_brow_h) / 2

    left_eye_ap = euclidean(lm(LEFT_EYE_TOP), lm(LEFT_EYE_BOTTOM)) / face_scale
    right_eye_ap = euclidean(lm(RIGHT_EYE_TOP), lm(RIGHT_EYE_BOTTOM)) / face_scale
    avg_eye_aperture = (left_eye_ap + right_eye_ap) / 2

    left_jaw_dist = euclidean(lm(LEFT_JAW), lm(CHIN))
    right_jaw_dist = euclidean(lm(RIGHT_JAW), lm(CHIN))
    jaw_asymmetry = abs(left_jaw_dist - right_jaw_dist) / face_scale

    return {
        'mar': mar,
        'lip_spread': lip_spread,
        'jaw_disp': jaw_disp,
        'brow_tension': brow_dist,
        'brow_height': avg_brow_height,
        'eye_aperture': avg_eye_aperture,
```

```
        'jaw_asymmetry': jaw_asymmetry,
    }


def compute_temporal_derivatives(features_array, fps):
    T, N = features_array.shape
    dt = 1.0 / fps

    velocity = np.zeros_like(features_array)
    velocity[1:] = (features_array[1:] - features_array[:-1]) / dt

    acceleration = np.zeros_like(features_array)
    acceleration[1:] = (velocity[1:] - velocity[:-1]) / dt

    abs_velocity = np.abs(velocity)

    window_size = max(int(0.3 * fps), 1)
    rolling_energy = np.zeros_like(features_array)
    for i in range(T):
        start = max(0, i - window_size // 2)
        end = min(T, i + window_size // 2 + 1)
        rolling_energy[i] = np.mean(abs_velocity[start:end], axis=0)

    tremor_window = max(int(0.3 * fps), 3)
    tremor = np.zeros_like(features_array)
    for i in range(T):
        start = max(0, i - tremor_window // 2)
        end = min(T, i + tremor_window // 2 + 1)
        if end - start > 1:
            tremor[i] = np.std(velocity[start:end], axis=0)

    return np.concatenate([
        features_array, velocity, acceleration, abs_velocity, rolling_energy, tremor
    ], axis=1)


def extract_video_features(video_path, target_fps=15):
    cap = cv2.VideoCapture(str(video_path))
    if not cap.isOpened():
        return None, 0

    actual_fps = cap.get(cv2.CAP_PROP_FPS)
    if actual_fps <= 0:
        actual_fps = 30.0

    w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    frame_skip = max(1, int(round(actual_fps / target_fps)))

    mp_face_mesh = mp.solutions.face_mesh
    face_mesh = mp_face_mesh.FaceMesh(
        static_image_mode=False,
        max_num_faces=1,
        refine_landmarks=True,
        min_detection_confidence=0.5,
        min_tracking_confidence=0.5,
    )

    frame_features = []
    frame_idx = 0
```

```
    missing_frames = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if frame_idx % frame_skip != 0:
            frame_idx += 1
            continue

        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = face_mesh.process(rgb)

        if results.multi_face_landmarks:
            landmarks = results.multi_face_landmarks[0].landmark
            feats = compute_frame_features(landmarks, w, h)
            frame_features.append(list(feats.values()))
        else:
            frame_features.append([np.nan] * 7)
            missing_frames += 1

        frame_idx += 1

    cap.release()
    face_mesh.close()

    if len(frame_features) == 0:
        return None, actual_fps

    features_array = np.array(frame_features, dtype=np.float64)

    for col in range(features_array.shape[1]):
        mask = np.isnan(features_array[:, col])
        if mask.any() and not mask.all():
            valid_idx = np.where(~mask)[0]
            features_array[mask, col] = np.interp(
                np.where(mask)[0], valid_idx, features_array[valid_idx, col]
            )
        elif mask.all():
            features_array[:, col] = 0.0

    for col in range(features_array.shape[1]):
        median = np.median(features_array[:, col])
        q75, q25 = np.percentile(features_array[:, col], [75, 25])
        iqr = q75 - q25
        if iqr < 1e-8:
            iqr = np.std(features_array[:, col])
        if iqr < 1e-8:
            iqr = 1.0
        features_array[:, col] = np.clip((features_array[:, col] - median) / iqr, -6, 6)

    effective_fps = actual_fps / frame_skip
    full_features = compute_temporal_derivatives(features_array, effective_fps)

    if missing_frames > 0:
        total = len(frame_features)
        pct = missing_frames / total * 100
        if pct > 30:
```

```
            tqdm.write(f"  [WARN] {video_path.name}: {pct:.0f}% frames without detected face")

    return full_features, effective_fps


FEATURE_NAMES_BASE = [
    'mar', 'lip_spread', 'jaw_disp', 'brow_tension',
    'brow_height', 'eye_aperture', 'jaw_asymmetry'
]
DERIVATIVE_SUFFIXES = ['_raw', '_vel', '_acc', '_abs_vel', '_roll_energy', '_tremor']


def get_all_feature_names():
    return [base + suffix for suffix in DERIVATIVE_SUFFIXES for base in FEATURE_NAMES_BASE]


def main():
    parser = argparse.ArgumentParser(description="Extract facial kinematics features from video files")
    parser.add_argument('--video_dir', type=str, required=True)
    parser.add_argument('--output_dir', type=str, required=True)
    parser.add_argument('--target_fps', type=int, default=15)
    parser.add_argument('--extensions', type=str, default='.mp4,.avi,.mov,.mkv')
    args = parser.parse_args()

    video_dir = Path(args.video_dir)
    output_dir = Path(args.output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    extensions = tuple(args.extensions.split(','))
    video_files = sorted([f for f in video_dir.iterdir() if f.suffix.lower() in extensions])

    print(f"\n{'='*60}")
    print(f"  [INFO] Facial Kinematics Extraction")
    print(f"  [INFO] Video directory: {video_dir} ({len(video_files)} files)")
    print(f"  [INFO] Target FPS: {args.target_fps}")
    print(f"  [INFO] Features per frame: {len(get_all_feature_names())}")
    print(f"{'='*60}\n")

    processed = 0
    for vf in tqdm(video_files, desc="Processing videos", colour='green'):
        npy_path = (output_dir / vf.stem).with_suffix('.npy')
        if npy_path.exists():
            processed += 1
            continue

        features, _ = extract_video_features(vf, args.target_fps)
        if features is not None:
            np.save(str(npy_path), features.astype(np.float32))
            processed += 1
            tqdm.write(f"  [OK] {vf.name}: {features.shape[0]} frames x {features.shape[1]} features")
        else:
            tqdm.write(f"  [ERROR] {vf.name}: failed to process")

    print(f"\n[DONE] Processed: {processed}/{len(video_files)}")
    print(f"[INFO] Saved to: {output_dir}")
```

```
250
251
252 if __name__ == "__main__":
253     main()
```

A.13. Обучение визуального классификатора (train_face_tcn.py)

```
1  """
2  Step 3: TCN classifier on visual kinematic features (Ablation).
3  Fixed window scale for 15 FPS (3.0s window = 45 frames).
4  """
5
6  import argparse
7  import json
8  import torch
9  import torch.optim as optim
10 from torch.utils.data import Dataset, DataLoader
11 import numpy as np
12 from pathlib import Path
13 from sklearn.model_selection import GroupKFold
14 from tqdm import tqdm
15
16 from utils import FocalLoss, compute_metrics, get_project_root
17 from train_tcn import StutterTCN
18
19
20 class VideoKinematicsDataset(Dataset):
21     def __init__(self, video_list, features_dir, labels_json,
22                  window_frames=45, stride_frames=15, desc="Loading"):
23         self.data, self.labels, self.masks, self.video_ids = [], [], [], []
24         features_dir = Path(features_dir)
25
26         with open(labels_json, 'r', encoding='utf-8') as f:
27             annotations = json.load(f)
28
29         for video_name in tqdm(video_list, desc=f"[{desc}]", leave=False):
30             npy_path = features_dir / f"{video_name}.npy"
31             if not npy_path.exists():
32                 continue
33
34             features = np.load(str(npy_path))
35             T, _ = features.shape
36
37             frame_labels = np.zeros(T, dtype=np.float32)
38             if video_name in annotations:
39                 for start, end in annotations[video_name]:
40                     frame_labels[max(0, int(start)):min(T, int(end))] = 1.0
41
42             for win_start in range(0, max(1, T - window_frames // 2), stride_frames):
43                 win_end = min(win_start + window_frames, T)
44                 segment = features[win_start:win_end]
45                 seg_labels = frame_labels[win_start:win_end]
46
47                 actual_len = segment.shape[0]
48                 if actual_len < window_frames:
```

```
                    pad_len = window_frames - actual_len
                    segment = np.pad(segment, ((0, pad_len), (0, 0)), mode='constant')
                    seg_labels = np.pad(seg_labels, (0, pad_len), mode='constant')
                    mask = np.concatenate([
                        np.ones(actual_len, dtype=np.float32),
                        np.zeros(pad_len, dtype=np.float32)
                    ])
                else:
                    mask = np.ones(window_frames, dtype=np.float32)

                stutter_ratio = seg_labels.sum() / mask.sum() if mask.sum() > 0 else 0
                binary_label = 1.0 if stutter_ratio >= 0.3 else 0.0

                self.data.append(segment)
                self.masks.append(mask)
                self.labels.append(binary_label)
                self.video_ids.append(video_name)

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return (
            torch.tensor(self.data[idx], dtype=torch.float32),
            torch.tensor(self.masks[idx], dtype=torch.float32),
            torch.tensor(self.labels[idx], dtype=torch.float32),
        )


def train_face_tcn(args):
    base_dir = get_project_root()
    features_dir = Path(args.features_dir)
    save_path = base_dir / "scripts" / "best_face_tcn.pth"

    print("\n" + "=" * 60)
    print("  [INFO] Face TCN Training (Visual Kinematics)")
    print(f"  [INFO] Features: {features_dir}")
    print(f"  [INFO] Window: {args.window_frames} frames ({args.window_frames/15:.1f}s),
    Stride: {args.stride_frames} frames")
    print("=" * 60)

    with open(args.labels_json, 'r', encoding='utf-8') as f:
        annotations = json.load(f)

    if args.patient_map_json:
        with open(args.patient_map_json, 'r', encoding='utf-8') as f:
            patient_map = json.load(f)
    else:
        patient_map = {v: v for v in annotations.keys()}

    video_list = sorted([v for v in annotations.keys() if (features_dir /
    f"{v}.npy").exists()])
    patients = [patient_map.get(v, v) for v in video_list]

    print(f"[DATA] Videos: {len(video_list)}, Unique patients: {len(set(patients))}")

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"[DEVICE] Compute device: {device}")

    sample = np.load(str(features_dir / f"{video_list[0]}.npy"))
```

```
    input_dim = sample.shape[1]
    print(f"[MODEL] Input dim: {input_dim}")

    n_folds = min(args.n_folds, len(set(patients)))
    gkf = GroupKFold(n_splits=n_folds)
    groups = np.array(patients)
    video_array = np.array(video_list)

    fold_results = []
    best_overall_roc = 0.0

    for fold, (train_idx, val_idx) in enumerate(gkf.split(video_array, groups=groups)):
        print(f"\n--- Fold {fold+1}/{n_folds} ---")
        train_videos = video_array[train_idx].tolist()
        val_videos = video_array[val_idx].tolist()
        print(f"  [SPLIT] Train: {train_videos}, Val: {val_videos}")

        train_dataset = VideoKinematicsDataset(
            train_videos, features_dir, args.labels_json,
            window_frames=args.window_frames, stride_frames=args.stride_frames,
            desc=f"Fold-{fold+1} Train"
        )
        val_dataset = VideoKinematicsDataset(
            val_videos, features_dir, args.labels_json,
            window_frames=args.window_frames, stride_frames=args.stride_frames,
            desc=f"Fold-{fold+1} Val"
        )

        train_loader = DataLoader(train_dataset, batch_size=args.batch_size,
    shuffle=True, num_workers=0)
        val_loader = DataLoader(val_dataset, batch_size=args.batch_size, shuffle=False,
    num_workers=0)

        model = StutterTCN(
            input_dim=input_dim, hidden_dim=128, num_blocks=3, kernel_size=5, dropout=0.3
        ).to(device)

        criterion = FocalLoss(alpha=0.25, gamma=2.0)
        optimizer = optim.AdamW(model.parameters(), lr=args.lr, weight_decay=1e-3)
        scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=args.epochs,
    eta_min=1e-6)

        best_fold_roc = 0.0

        for epoch in range(args.epochs):
            model.train()
            for emb, mask, labels in train_loader:
                emb, mask, labels = emb.to(device), mask.to(device), labels.to(device)
                optimizer.zero_grad()
                logits = model(emb, mask)
                loss = criterion(logits, labels)
                loss.backward()
                torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                optimizer.step()

            scheduler.step()

            model.eval()
            all_probs, all_preds, all_true = [], [], []
            with torch.no_grad():
```

```
                for emb, mask, labels in val_loader:
                    emb, mask = emb.to(device), mask.to(device)
                    logits = model(emb, mask)
                    probs = torch.sigmoid(logits)
                    all_probs.extend(probs.cpu().numpy())
                    all_preds.extend((probs > 0.5).float().cpu().numpy())
                    all_true.extend(labels.numpy())

            if len(set(all_true)) > 1:
                metrics = compute_metrics(all_true, all_preds, all_probs)
                current_roc = metrics.get('roc_auc', 0.0)
            else:
                current_roc = 0.5

            if current_roc > best_fold_roc:
                best_fold_roc = current_roc
                if current_roc > best_overall_roc:
                    best_overall_roc = current_roc
                    torch.save({
                        'model_state_dict': model.state_dict(),
                        'fold': fold,
                        'epoch': epoch,
                        'metrics': compute_metrics(all_true, all_preds, all_probs) if
    len(set(all_true)) > 1 else {},
                        'input_dim': input_dim,
                    }, save_path)

        print(f"  [INFO] Fold {fold+1} best ROC-AUC: {best_fold_roc:.4f}")
        fold_results.append({'fold': fold + 1, 'best_roc_auc': best_fold_roc})

    print(f"\n{'='*60}")
    print(f"  [DONE] Training complete")
    if fold_results:
        print(f"  [RESULTS] Mean ROC-AUC across folds: {np.mean([r['best_roc_auc'] for r
    in fold_results]):.4f}")
        print(f"  [RESULTS] Best overall ROC-AUC: {best_overall_roc:.4f}")
    if save_path.exists():
        print(f"  [SAVE] Model saved to: {save_path}")
    print(f"{'='*60}")


def parse_args():
    parser = argparse.ArgumentParser(description="Train Face TCN on visual kinematics
    features")
    parser.add_argument('--features_dir', type=str, required=True)
    parser.add_argument('--labels_json', type=str, required=True)
    parser.add_argument('--patient_map_json', type=str, default=None)
    parser.add_argument('--window_frames', type=int, default=45)
    parser.add_argument('--stride_frames', type=int, default=15)
    parser.add_argument('--batch_size', type=int, default=16)
    parser.add_argument('--epochs', type=int, default=40)
    parser.add_argument('--lr', type=float, default=1e-3)
    parser.add_argument('--n_folds', type=int, default=3)
    return parser.parse_args()


if __name__ == "__main__":
    args = parse_args()
    train_face_tcn(args)
```

A.14. Архитектура и обучение мультимодальной сети (train_deep_fusion.py)

```
"""
Step 4: Deep Fusion - multimodal TCN with early/deep audio-visual fusion.
Synchronized label interpolation across 15 FPS video and 50 FPS audio.
"""

import argparse
import json
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import numpy as np
from pathlib import Path
from sklearn.model_selection import GroupKFold
from tqdm import tqdm

from utils import FocalLoss, compute_metrics, get_project_root


class ResidualBlock1D(nn.Module):
    def __init__(self, channels, kernel_size=5, dilation=1, dropout=0.2):
        super().__init__()
        padding = (kernel_size - 1) * dilation // 2
        self.conv1 = nn.Conv1d(channels, channels, kernel_size, padding=padding,
    dilation=dilation)
        self.bn1 = nn.BatchNorm1d(channels)
        self.conv2 = nn.Conv1d(channels, channels, kernel_size=1)
        self.bn2 = nn.BatchNorm1d(channels)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        residual = x
        out = self.dropout(F.relu(self.bn1(self.conv1(x))))
        out = self.bn2(self.conv2(out))
        if out.shape[-1] != residual.shape[-1]:
            out = out[..., :residual.shape[-1]]
        return F.relu(out + residual)


class StreamEncoder(nn.Module):
    def __init__(self, input_dim, hidden_dim=256, num_blocks=4, kernel_size=5,
    dropout=0.3):
        super().__init__()
        self.proj = nn.Sequential(
            nn.Conv1d(input_dim, hidden_dim, kernel_size=1),
            nn.BatchNorm1d(hidden_dim),
            nn.ReLU(),
        )
        dilations = [2 ** i for i in range(num_blocks)]
        self.blocks = nn.ModuleList([
            ResidualBlock1D(hidden_dim, kernel_size, dilation=d, dropout=dropout)
            for d in dilations
        ])

```

```
    def forward(self, x):
        x = self.proj(x)
        for block in self.blocks:
            x = block(x)
        return x


class DeepFusionTCN(nn.Module):
    def __init__(self, audio_dim=1024, video_dim=42, hidden_dim=256,
                 audio_blocks=4, video_blocks=3, fusion_blocks=2,
                 kernel_size=5, dropout=0.3, modality_dropout=0.2):
        super().__init__()
        self.modality_dropout = modality_dropout

        self.audio_encoder = StreamEncoder(audio_dim, hidden_dim, audio_blocks,
    kernel_size, dropout)
        self.video_encoder = StreamEncoder(video_dim, hidden_dim, video_blocks,
    kernel_size, dropout)

        fusion_dim = hidden_dim * 2
        self.fusion_proj = nn.Sequential(
            nn.Conv1d(fusion_dim, hidden_dim, kernel_size=1),
            nn.BatchNorm1d(hidden_dim),
            nn.ReLU(),
        )
        dilations = [2 ** i for i in range(fusion_blocks)]
        self.fusion_blocks = nn.ModuleList([
            ResidualBlock1D(hidden_dim, kernel_size, dilation=d, dropout=dropout)
            for d in dilations
        ])

        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim // 2, 1),
        )

    def forward(self, audio, video, mask=None):
        audio = audio.permute(0, 2, 1)
        video = video.permute(0, 2, 1)

        audio_feat = self.audio_encoder(audio)
        video_feat = self.video_encoder(video)

        if self.training and self.modality_dropout > 0:
            r = torch.rand(1).item()
            if r < self.modality_dropout / 2:
                audio_feat = torch.zeros_like(audio_feat)
            elif r < self.modality_dropout:
                video_feat = torch.zeros_like(video_feat)

        fused = torch.cat([audio_feat, video_feat], dim=1)
        fused = self.fusion_proj(fused)
        for block in self.fusion_blocks:
            fused = block(fused)

        fused = fused.permute(0, 2, 1)
        if mask is not None:
            mask_exp = mask.unsqueeze(-1)
```

```
            fused = (fused * mask_exp).sum(dim=1) / mask_exp.sum(dim=1).clamp(min=1)
        else:
            fused = fused.mean(dim=1)

        return self.classifier(fused).squeeze(-1)


class MultimodalDataset(Dataset):
    def __init__(self, video_list, audio_dir, video_dir, labels_json,
                 target_len=150, stride=50, desc="Loading"):
        self.data_audio, self.data_video, self.labels, self.masks = [], [], [], []
        audio_dir = Path(audio_dir)
        video_dir = Path(video_dir)

        with open(labels_json, 'r', encoding='utf-8') as f:
            annotations = json.load(f)

        for vname in tqdm(video_list, desc=f"[{desc}]", leave=False):
            audio_path = audio_dir / f"{vname}.npy"
            video_path = video_dir / f"{vname}.npy"

            if not audio_path.exists() or not video_path.exists():
                continue

            audio_feat = np.load(str(audio_path))
            video_feat = np.load(str(video_path))

            orig_v_len = video_feat.shape[0]
            raw_labels = np.zeros(orig_v_len, dtype=np.float32)
            if vname in annotations:
                for start, end in annotations[vname]:
                    raw_labels[max(0, int(start)):min(orig_v_len, int(end))] = 1.0

            T_target = max(audio_feat.shape[0], video_feat.shape[0])
            if audio_feat.shape[0] != T_target:
                audio_feat = self._resample(audio_feat, T_target)

            if video_feat.shape[0] != T_target:
                video_feat = self._resample(video_feat, T_target)
                x_src = np.linspace(0, 1, orig_v_len)
                x_tgt = np.linspace(0, 1, T_target)
                interp_labels = np.interp(x_tgt, x_src, raw_labels)
                frame_labels = (interp_labels >= 0.5).astype(np.float32)
            else:
                frame_labels = raw_labels

            T = T_target
            for ws in range(0, max(1, T - target_len // 2), stride):
                we = min(ws + target_len, T)
                a_seg = audio_feat[ws:we]
                v_seg = video_feat[ws:we]
                l_seg = frame_labels[ws:we]

                actual = a_seg.shape[0]
                if actual < target_len:
                    pad = target_len - actual
                    a_seg = np.pad(a_seg, ((0, pad), (0, 0)))
                    v_seg = np.pad(v_seg, ((0, pad), (0, 0)))
                    l_seg = np.pad(l_seg, (0, pad))
```

```
                    mask = np.concatenate([np.ones(actual),
    np.zeros(pad)]).astype(np.float32)
                else:
                    mask = np.ones(target_len, dtype=np.float32)

                stutter_ratio = l_seg.sum() / mask.sum() if mask.sum() > 0 else 0
                binary_label = 1.0 if stutter_ratio >= 0.3 else 0.0

                self.data_audio.append(a_seg.astype(np.float32))
                self.data_video.append(v_seg.astype(np.float32))
                self.masks.append(mask)
                self.labels.append(binary_label)

    @staticmethod
    def _resample(arr, target_len):
        T_src, D = arr.shape
        x_src = np.linspace(0, 1, T_src)
        x_tgt = np.linspace(0, 1, target_len)
        result = np.zeros((target_len, D), dtype=arr.dtype)
        for d in range(D):
            result[:, d] = np.interp(x_tgt, x_src, arr[:, d])
        return result

    def __len__(self):
        return len(self.data_audio)

    def __getitem__(self, idx):
        return (
            torch.tensor(self.data_audio[idx]),
            torch.tensor(self.data_video[idx]),
            torch.tensor(self.masks[idx]),
            torch.tensor(self.labels[idx]),
        )


def train_fusion(args):
    base_dir = get_project_root()
    save_path = base_dir / "scripts" / f"best_deep_fusion_{args.encoder_name}.pth"

    print("\n" + "=" * 60)
    print("  [INFO] Deep Fusion Training (Audio + Video)")
    print(f"  [INFO] Encoder: {args.encoder_name}")
    print(f"  [INFO] Window: {args.window_frames} frames ({args.window_frames/50:.1f}s),
    Stride: {args.stride_frames} frames")
    print("=" * 60)

    with open(args.labels_json, 'r', encoding='utf-8') as f:
        annotations = json.load(f)

    if args.patient_map_json:
        with open(args.patient_map_json, 'r', encoding='utf-8') as f:
            patient_map = json.load(f)
    else:
        patient_map = {v: v for v in annotations.keys()}

    audio_dir = Path(args.audio_dir)
    video_dir = Path(args.video_dir)

    video_list = sorted([
        v for v in annotations.keys()
```

```
        if (audio_dir / f"{v}.npy").exists() and (video_dir / f"{v}.npy").exists()
    ])
    patients = [patient_map.get(v, v) for v in video_list]

    print(f"[DATA] Videos with both modalities: {len(video_list)}")
    print(f"[DATA] Unique patients: {len(set(patients))}")

    sample_a = np.load(str(audio_dir / f"{video_list[0]}.npy"))
    sample_v = np.load(str(video_dir / f"{video_list[0]}.npy"))
    audio_dim = sample_a.shape[1]
    video_dim = sample_v.shape[1]
    print(f"[MODEL] Audio dim: {audio_dim}, Video dim: {video_dim}")

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"[DEVICE] Compute device: {device}")

    n_folds = min(args.n_folds, len(set(patients)))
    gkf = GroupKFold(n_splits=n_folds)
    groups = np.array(patients)
    video_array = np.array(video_list)

    fold_results = []
    best_overall_roc = 0.0

    for fold, (train_idx, val_idx) in enumerate(gkf.split(video_array, groups=groups)):
        print(f"\n--- Fold {fold+1}/{n_folds} ---")
        train_vids = video_array[train_idx].tolist()
        val_vids = video_array[val_idx].tolist()
        print(f"  [SPLIT] Train: {train_vids}, Val: {val_vids}")

        train_dataset = MultimodalDataset(
            train_vids, audio_dir, video_dir, args.labels_json,
            target_len=args.window_frames, stride=args.stride_frames,
            desc=f"F{fold+1} Train"
        )
        val_dataset = MultimodalDataset(
            val_vids, audio_dir, video_dir, args.labels_json,
            target_len=args.window_frames, stride=args.stride_frames,
            desc=f"F{fold+1} Val"
        )

        train_loader = DataLoader(train_dataset, batch_size=args.batch_size,
    shuffle=True, num_workers=0)
        val_loader = DataLoader(val_dataset, batch_size=args.batch_size, shuffle=False,
    num_workers=0)

        model = DeepFusionTCN(
            audio_dim=audio_dim, video_dim=video_dim,
            hidden_dim=args.hidden_dim, modality_dropout=args.modality_dropout,
        ).to(device)

        criterion = FocalLoss(alpha=0.25, gamma=2.0)
        optimizer = optim.AdamW(model.parameters(), lr=args.lr, weight_decay=1e-3)
        scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=args.epochs,
    eta_min=1e-6)

        best_fold_roc = 0.0

        for epoch in range(args.epochs):
            model.train()
```

```
            for audio, video, mask, labels in train_loader:
                audio, video, mask, labels = audio.to(device), video.to(device),
    mask.to(device), labels.to(device)
                optimizer.zero_grad()
                logits = model(audio, video, mask)
                loss = criterion(logits, labels)
                loss.backward()
                torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                optimizer.step()

            scheduler.step()

            model.eval()
            all_probs, all_preds, all_true = [], [], []
            with torch.no_grad():
                for audio, video, mask, labels in val_loader:
                    audio, video, mask = audio.to(device), video.to(device),
    mask.to(device)
                    logits = model(audio, video, mask)
                    probs = torch.sigmoid(logits)
                    all_probs.extend(probs.cpu().numpy())
                    all_preds.extend((probs > 0.5).float().cpu().numpy())
                    all_true.extend(labels.numpy())

            if len(set(all_true)) > 1:
                metrics = compute_metrics(all_true, all_preds, all_probs)
                current_roc = metrics.get('roc_auc', 0.0)
            else:
                current_roc = 0.5

            if current_roc > best_fold_roc:
                best_fold_roc = current_roc
                if current_roc > best_overall_roc:
                    best_overall_roc = current_roc
                    torch.save({
                        'model_state_dict': model.state_dict(),
                        'fold': fold,
                        'epoch': epoch,
                        'metrics': compute_metrics(all_true, all_preds, all_probs) if
    len(set(all_true)) > 1 else {},
                        'audio_dim': audio_dim,
                        'video_dim': video_dim,
                        'hidden_dim': args.hidden_dim,
                    }, save_path)

        print(f"  [INFO] Fold {fold+1} best ROC-AUC: {best_fold_roc:.4f}")
        fold_results.append(best_fold_roc)

    print(f"\n{'='*60}")
    print(f"  [DONE] Deep Fusion training complete ({args.encoder_name})")
    if fold_results:
        print(f"  [RESULTS] Mean ROC-AUC: {np.mean(fold_results):.4f} +/-
    {np.std(fold_results):.4f}")
        print(f"  [RESULTS] Best ROC-AUC: {best_overall_roc:.4f}")
    if save_path.exists():
        print(f"  [SAVE] Model saved to: {save_path}")
    print(f"{'='*60}")


def parse_args():
```

```
    parser = argparse.ArgumentParser(description="Train Deep Fusion TCN")
    parser.add_argument('--audio_dir', type=str, required=True)
    parser.add_argument('--video_dir', type=str, required=True)
    parser.add_argument('--labels_json', type=str, required=True)
    parser.add_argument('--patient_map_json', type=str, default=None)
    parser.add_argument('--window_frames', type=int, default=150)
    parser.add_argument('--stride_frames', type=int, default=50)
    parser.add_argument('--hidden_dim', type=int, default=256)
    parser.add_argument('--modality_dropout', type=float, default=0.2)
    parser.add_argument('--batch_size', type=int, default=16)
    parser.add_argument('--epochs', type=int, default=40)
    parser.add_argument('--lr', type=float, default=3e-4)
    parser.add_argument('--n_folds', type=int, default=3)
    parser.add_argument('--encoder_name', type=str, default='whisper',
                        choices=['wav2vec2', 'wavlm', 'whisper'])
    return parser.parse_args()


if __name__ == "__main__":
    args = parse_args()
    train_fusion(args)
```

A.15. Скрипт кросс-языковой валидации и визуализации (evaluate_crosslingual.py)

```
"""
Step 4: Cross-lingual validation - evaluating all models on Russian data.
Run: python scripts/evaluate_crosslingual.py --russian_audio_dir <> --russian_video_dir
    <> --labels_json <> --output_dir <>
"""

import argparse
import json
import torch
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from pathlib import Path
from tabulate import tabulate

from utils import compute_metrics, get_project_root
from train_tcn import StutterTCN
from train_deep_fusion import DeepFusionTCN


def load_frame_labels(labels_json, video_name, T):
    with open(labels_json, 'r') as f:
        annotations = json.load(f)

    frame_labels = np.zeros(T, dtype=np.float32)
    if video_name in annotations:
        for start, end in annotations[video_name]:
            s, e = max(0, int(start)), min(T, int(end))
            frame_labels[s:e] = 1.0
    return frame_labels
```

```
31
32
33  def evaluate_audio_tcn(audio_dir, labels_json, video_list, device,
        encoder_name='wav2vec2'):
34      base_dir = get_project_root()
35      ckpt_path = base_dir / "scripts" / f"best_tcn_{encoder_name}.pth"
36
37      if not ckpt_path.exists():
38          return None, None
39
40      checkpoint = torch.load(ckpt_path, map_location=device, weights_only=False)
41      saved_args = checkpoint.get('args', {})
42
43      model = StutterTCN(
44          input_dim=saved_args.get('input_dim', 1024),
45          hidden_dim=saved_args.get('hidden_dim', 256),
46          num_blocks=saved_args.get('num_blocks', 4),
47          kernel_size=saved_args.get('kernel_size', 5),
48          dropout=0.0,
49      ).to(device)
50      model.load_state_dict(checkpoint['model_state_dict'])
51      model.eval()
52
53      all_true, all_probs = [], []
54      per_video_results = {}
55      window_len = saved_args.get('max_seq_len', 150)
56      stride = window_len // 2
57      audio_dir = Path(audio_dir)
58
59      for vname in video_list:
60          npy_path = audio_dir / f"{vname}.npy"
61          if not npy_path.exists():
62              continue
63
64          features = np.load(str(npy_path))
65          T = features.shape[0]
66          frame_labels = load_frame_labels(labels_json, vname, T)
67
68          frame_scores = np.zeros(T)
69          frame_counts = np.zeros(T)
70
71          with torch.no_grad():
72              for ws in range(0, T, stride):
73                  we = min(ws + window_len, T)
74                  segment = features[ws:we]
75                  actual = segment.shape[0]
76
77                  if actual < window_len:
78                      segment = np.pad(segment, ((0, window_len - actual), (0, 0)))
79                      mask = np.concatenate([np.ones(actual), np.zeros(window_len -
        actual)])
80                  else:
81                      mask = np.ones(window_len)
82
83                  seg_t = torch.tensor(segment,
        dtype=torch.float32).unsqueeze(0).to(device)
84                  mask_t = torch.tensor(mask, dtype=torch.float32).unsqueeze(0).to(device)
85
86                  logit = model(seg_t, mask_t)
87                  prob = torch.sigmoid(logit).item()
```

```
                frame_scores[ws:we] += prob
                frame_counts[ws:we] += 1

        frame_counts = np.maximum(frame_counts, 1)
        frame_probs = frame_scores / frame_counts

        all_true.extend(frame_labels.tolist())
        all_probs.extend(frame_probs.tolist())
        per_video_results[vname] = {'labels': frame_labels, 'probs': frame_probs}

    all_preds = [1 if p > 0.5 else 0 for p in all_probs]
    metrics = compute_metrics(all_true, all_preds, all_probs)
    return metrics, per_video_results


def evaluate_deep_fusion(audio_dir, video_dir, labels_json, video_list, device,
    encoder_name='whisper'):
    base_dir = get_project_root()
    ckpt_path = base_dir / "scripts" / f"best_deep_fusion_{encoder_name}.pth"
    if not ckpt_path.exists():
        ckpt_path = base_dir / "scripts" / "best_deep_fusion.pth"
    if not ckpt_path.exists():
        return None, None

    checkpoint = torch.load(ckpt_path, map_location=device, weights_only=False)

    model = DeepFusionTCN(
        audio_dim=checkpoint['audio_dim'],
        video_dim=checkpoint['video_dim'],
        hidden_dim=checkpoint.get('hidden_dim', 256),
        modality_dropout=0.0,
    ).to(device)
    model.load_state_dict(checkpoint['model_state_dict'])
    model.eval()

    audio_dir = Path(audio_dir)
    video_dir = Path(video_dir)

    all_true, all_probs = [], []
    per_video_results = {}
    window_len = 450
    stride = 150

    for vname in video_list:
        a_path = audio_dir / f"{vname}.npy"
        v_path = video_dir / f"{vname}.npy"
        if not a_path.exists() or not v_path.exists():
            continue

        audio_feat = np.load(str(a_path))
        video_feat = np.load(str(v_path))

        T = max(audio_feat.shape[0], video_feat.shape[0])
        if audio_feat.shape[0] != T:
            audio_feat = _resample_array(audio_feat, T)
        if video_feat.shape[0] != T:
            video_feat = _resample_array(video_feat, T)

        frame_labels = load_frame_labels(labels_json, vname, T)
```

```
        frame_scores = np.zeros(T)
        frame_counts = np.zeros(T)

        with torch.no_grad():
            for ws in range(0, T, stride):
                we = min(ws + window_len, T)
                a_seg = audio_feat[ws:we]
                v_seg = video_feat[ws:we]
                actual = a_seg.shape[0]

                if actual < window_len:
                    pad = window_len - actual
                    a_seg = np.pad(a_seg, ((0, pad), (0, 0)))
                    v_seg = np.pad(v_seg, ((0, pad), (0, 0)))
                    mask = np.concatenate([np.ones(actual), np.zeros(pad)])
                else:
                    mask = np.ones(window_len)

                a_t = torch.tensor(a_seg, dtype=torch.float32).unsqueeze(0).to(device)
                v_t = torch.tensor(v_seg, dtype=torch.float32).unsqueeze(0).to(device)
                m_t = torch.tensor(mask, dtype=torch.float32).unsqueeze(0).to(device)

                logit = model(a_t, v_t, m_t)
                prob = torch.sigmoid(logit).item()

                frame_scores[ws:we] += prob
                frame_counts[ws:we] += 1

        frame_counts = np.maximum(frame_counts, 1)
        frame_probs = frame_scores / frame_counts

        all_true.extend(frame_labels.tolist())
        all_probs.extend(frame_probs.tolist())
        per_video_results[vname] = {'labels': frame_labels, 'probs': frame_probs}

    all_preds = [1 if p > 0.5 else 0 for p in all_probs]
    metrics = compute_metrics(all_true, all_preds, all_probs)
    return metrics, per_video_results


def _resample_array(arr, target_len):
    T_src, D = arr.shape
    x_src = np.linspace(0, 1, T_src)
    x_tgt = np.linspace(0, 1, target_len)
    result = np.zeros((target_len, D), dtype=arr.dtype)
    for d in range(D):
        result[:, d] = np.interp(x_tgt, x_src, arr[:, d])
    return result


def plot_timeline(video_name, labels, probs, threshold=0.5, save_path=None):
    fig, ax = plt.subplots(1, 1, figsize=(14, 3))
    T = len(labels)
    x = np.arange(T)

    in_event = False
    for i in range(T):
        if labels[i] > 0.5 and not in_event:
            start = i
            in_event = True
```

```
        elif (labels[i] <= 0.5 or i == T - 1) and in_event:
            ax.axvspan(start, i, alpha=0.3, color='green')
            in_event = False

    preds = (np.array(probs) > threshold).astype(float)
    in_det = False
    for i in range(T):
        if preds[i] > 0.5 and not in_det:
            start = i
            in_det = True
        elif (preds[i] <= 0.5 or i == T - 1) and in_det:
            ax.axvspan(start, i, alpha=0.2, color='red', hatch='///')
            in_det = False

    ax.plot(x, probs, color='blue', linewidth=0.8, alpha=0.8)
    ax.axhline(y=threshold, color='gray', linestyle='--', alpha=0.5, linewidth=0.5)

    ax.set_xlim(0, T)
    ax.set_ylim(0, 1)
    ax.set_xlabel('Frame')
    ax.set_ylabel('Score')
    ax.set_title(f'{video_name}', fontsize=11)

    handles = [
        mpatches.Patch(color='green', alpha=0.3, label='Annotation'),
        mpatches.Patch(color='red', alpha=0.2, hatch='///', label='Detection'),
    ]
    ax.legend(handles=handles, loc='upper right', fontsize=8)
    plt.tight_layout()

    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
        plt.close()
    else:
        plt.show()


def main():
    parser = argparse.ArgumentParser(description="Cross-lingual evaluation on Russian data")
    parser.add_argument('--russian_audio_dir', type=str, default=None,
                        help="Optional override; by default uses data/russian_seq_<encoder>")
    parser.add_argument('--russian_video_dir', type=str, default='data/russian_face_kinematics')
    parser.add_argument('--labels_json', type=str, default='data/russian_labels.json')
    parser.add_argument('--output_dir', type=str, default='results/plots')
    parser.add_argument('--encoders', type=str, default='wav2vec2,wavlm,whisper')
    args = parser.parse_args()

    base_dir = get_project_root()
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    with open(args.labels_json, 'r', encoding='utf-8') as f:
        annotations = json.load(f)
    video_list = sorted(annotations.keys())

    print("\n" + "=" * 70)
    print("  [INFO] Cross-Lingual & Multimodal Validation (RU)")
    print(f"  [INFO] Videos: {len(video_list)}")
```

```
    print("=" * 70)

    results = []
    all_per_video = {}

    for enc in [e.strip() for e in args.encoders.split(',')]:
        enc_audio_dir = Path(args.russian_audio_dir) if args.russian_audio_dir else
    (base_dir / "data" / f"russian_seq_{enc}")

        print(f"\n[EVAL] Audio TCN ({enc})...")
        metrics, per_vid = evaluate_audio_tcn(enc_audio_dir, args.labels_json,
    video_list, device, enc)
        if metrics:
            metrics['method'] = f'Audio TCN ({enc})'
            results.append(metrics)
            all_per_video[f'TCN_{enc}'] = per_vid

        if args.russian_video_dir and Path(args.russian_video_dir).exists():
            print(f"[EVAL] Deep Fusion ({enc} + Face Kinematics)...")
            df_metrics, df_per_vid = evaluate_deep_fusion(
                enc_audio_dir, args.russian_video_dir, args.labels_json, video_list,
    device, enc
            )
            if df_metrics:
                df_metrics['method'] = f'Deep Fusion ({enc} + Video)'
                results.append(df_metrics)
                all_per_video[f'DeepFusion_{enc}'] = df_per_vid


if __name__ == "__main__":
    main()
```

A.16. Модуль инспекции видео и предварительной детекции (auto_inspect_videos.py)

```
"""
Auto-inspector & pre-labeler for Russian stuttering videos using system FFmpeg.
Run: python scripts/auto_inspect_videos.py
"""

import json
import subprocess
import cv2
import torch
import numpy as np
import soundfile as sf
import mediapipe as mp
from pathlib import Path
from transformers import WhisperFeatureExtractor, WhisperModel

from train_tcn import StutterTCN
from utils import get_project_root


def extract_wav_system_ffmpeg(video_files, audio_dir: Path):
    audio_dir.mkdir(parents=True, exist_ok=True)
```

```
    print("\n[1/3] Extracting 16kHz mono WAV via FFmpeg...")
    for vf in video_files:
        out_wav = audio_dir / f"{vf.stem}.wav"
        cmd = [
            "ffmpeg", "-y", "-i", str(vf),
            "-vn", "-acodec", "pcm_s16le", "-ar", "16000", "-ac", "1",
            str(out_wav)
        ]
        subprocess.run(cmd, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
        print(f"  [OK] {out_wav.name}")


def check_face_visibility(video_path: Path):
    cap = cv2.VideoCapture(str(video_path))
    fps = cap.get(cv2.CAP_PROP_FPS) or 30.0
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    duration = total_frames / fps
    step = max(1, int(round(fps / 5)))

    mp_face = mp.solutions.face_mesh.FaceMesh(static_image_mode=False, max_num_faces=1)
    sec_status = {}
    idx = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        if idx % step == 0:
            sec = int(idx / fps)
            rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            has_face = mp_face.process(rgb).multi_face_landmarks is not None
            sec_status.setdefault(sec, []).append(has_face)
        idx += 1
    cap.release()
    mp_face.close()

    bad_seconds = [s for s, vals in sec_status.items() if sum(vals) / len(vals) < 0.5]
    face_rate = 100.0 * (1.0 - len(bad_seconds) / max(len(sec_status), 1))
    return duration, face_rate, bad_seconds


def suggest_stutter_intervals(wav_path: Path, fe, whisper_model, tcn_model, device):
    speech, sr = sf.read(str(wav_path))
    win_samples = int(3.0 * sr)
    hop_samples = int(0.5 * sr)
    duration_s = len(speech) / sr

    sec_probs = np.zeros(int(np.ceil(duration_s * 10)))
    sec_counts = np.zeros_like(sec_probs)

    with torch.no_grad():
        for start in range(0, max(1, len(speech) - int(0.5 * sr)), hop_samples):
            chunk = speech[start:start + win_samples]
            inputs = fe(
                chunk, sampling_rate=16000,
                return_tensors="pt", padding="max_length", max_length=16000 * 30
            )
            hidden =
    whisper_model.encoder(inputs.input_features.to(device)).last_hidden_state
            real_frames = min(int(np.ceil(len(chunk) / 320)), 150)
            emb = hidden[:, :real_frames, :]
```

```
            if real_frames < 150:
                pad = torch.zeros((1, 150 - real_frames, 768), device=device)
                emb = torch.cat([emb, pad], dim=1)
            mask = torch.zeros((1, 150), device=device)
            mask[:, :real_frames] = 1.0

            prob = torch.sigmoid(tcn_model(emb, mask)).item()
            t0 = int((start / sr) * 10)
            t1 = min(len(sec_probs), int(((start + len(chunk)) / sr) * 10))
            sec_probs[t0:t1] += prob
            sec_counts[t0:t1] += 1

    avg_probs = sec_probs / np.maximum(sec_counts, 1)
    is_stutter = avg_probs >= 0.55

    intervals = []
    in_seg = False
    for i, val in enumerate(is_stutter):
        if val and not in_seg:
            s_start = i / 10.0
            in_seg = True
        elif not val and in_seg:
            if (i / 10.0) - s_start >= 0.5:
                intervals.append([round(s_start, 1), round(i / 10.0, 1)])
            in_seg = False
    if in_seg and (len(is_stutter) / 10.0 - s_start >= 0.5):
        intervals.append([round(s_start, 1), round(len(is_stutter) / 10.0, 1)])

    return intervals


def main():
    base_dir = get_project_root()
    video_dir = base_dir / "data" / "russian_videos"
    audio_dir = base_dir / "data" / "russian_audio"

    video_files = sorted(video_dir.glob("ru_*.mp4"))
    if not video_files:
        print(f"[ERROR] No ru_*.mp4 files found in {video_dir}")
        return

    extract_wav_system_ffmpeg(video_files, audio_dir)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"\n[2/3] Loading Whisper-small & best_tcn_whisper.pth on {device}...")
    fe = WhisperFeatureExtractor.from_pretrained("openai/whisper-small")
    whisper_model =
    WhisperModel.from_pretrained("openai/whisper-small").to(device).eval()

    ckpt = torch.load(base_dir / "scripts" / "best_tcn_whisper.pth",
    map_location=device, weights_only=False)
    tcn_model = StutterTCN(input_dim=768, hidden_dim=256, num_blocks=4, kernel_size=5,
    dropout=0.0).to(device)
    tcn_model.load_state_dict(ckpt["model_state_dict"])
    tcn_model.eval()

    print("\n[3/3] Inspecting videos & detecting stuttering intervals:\n" + "=" * 70)
    suggested_seconds = {}

    for vf in video_files:
```

```
        dur, face_rate, bad_secs = check_face_visibility(vf)
        wav_path = audio_dir / f"{vf.stem}.wav"
        intervals = suggest_stutter_intervals(wav_path, fe, whisper_model, tcn_model,
    device)
        suggested_seconds[vf.stem] = intervals

        trim_note = "OK (no trim needed)" if not bad_secs else f"No face at sec:
    {bad_secs}"
        print(f"Video: {vf.name} | Duration: {dur:.1f}s | Face visible: {face_rate:.1f}%
    ({trim_note})")
        print(f"  Suggested stutter intervals (sec): {intervals}\n")

    print("=" * 70)
    print("ANNOTATIONS_SECONDS =", json.dumps(suggested_seconds, indent=4))


if __name__ == "__main__":
    main()
```

A.17. Автоматическая обрезка заставок и синхронизация разметки (trim_intros.py)

```
"""
Trim intros from Russian videos, re-extract audio, and shift stutter labels.
Run: python scripts/trim_intros.py
"""

import json
import shutil
import subprocess
from pathlib import Path

CUT_OFFSETS = {
    "ru_01": 6.0,
    "ru_02": 5.0,
    "ru_03": 6.0,
    "ru_04": 6.0,
    "ru_05": 5.0,
    "ru_06": 6.0,
}

RAW_INTERVALS = {
    "ru_01": [[7.0, 12.0], [16.5, 27.0], [30.5, 41.0], [47.0, 49.0]],
    "ru_02": [[4.5, 9.0], [10.5, 45.5], [47.5, 78.0], [83.5, 120.5]],
    "ru_03": [[7.5, 24.5], [26.5, 29.5], [32.5, 47.0]],
    "ru_04": [[6.0, 28.0], [29.5, 36.5]],
    "ru_05": [[6.0, 12.0], [13.5, 46.5]],
    "ru_06": [[8.0, 10.5], [15.0, 18.0], [18.5, 22.5], [33.0, 42.0], [45.5, 50.0]],
}

TARGET_FPS = 15


def main():
    base_dir = Path(__file__).resolve().parent.parent
    video_dir = base_dir / "data" / "russian_videos"
```

```
    backup_dir = base_dir / "data" / "russian_videos_raw"
    audio_dir = base_dir / "data" / "russian_audio"
    labels_path = base_dir / "data" / "russian_labels.json"

    backup_dir.mkdir(parents=True, exist_ok=True)
    audio_dir.mkdir(parents=True, exist_ok=True)

    print("\n[1/3] Backing up original videos to data/russian_videos_raw/...")
    for vf in sorted(video_dir.glob("ru_*.mp4")):
        backup_file = backup_dir / vf.name
        if not backup_file.exists():
            shutil.copy2(vf, backup_file)

    print("\n[2/3] Trimming intros via FFmpeg (video + 16kHz mono audio)...")
    for stem, offset in CUT_OFFSETS.items():
        src_video = backup_dir / f"{stem}.mp4"
        out_video = video_dir / f"{stem}.mp4"
        out_audio = audio_dir / f"{stem}.wav"

        cmd_video = [
            "ffmpeg", "-y", "-ss", str(offset), "-i", str(src_video),
            "-c:v", "libx264", "-preset", "veryfast", "-crf", "20",
            "-c:a", "aac", str(out_video)
        ]
        subprocess.run(cmd_video, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

        cmd_audio = [
            "ffmpeg", "-y", "-ss", str(offset), "-i", str(src_video),
            "-vn", "-acodec", "pcm_s16le", "-ar", "16000", "-ac", "1",
            str(out_audio)
        ]
        subprocess.run(cmd_audio, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
        print(f"  [OK] {stem}: cut {offset}s intro -> video & audio updated")

    print("\n[3/3] Synchronizing stutter intervals into frame indices...")
    shifted_seconds = {}
    frame_labels = {}

    for stem, intervals in RAW_INTERVALS.items():
        offset = CUT_OFFSETS[stem]
        shifted_list = []
        frame_list = []

        for start_s, end_s in intervals:
            new_start = max(0.0, round(start_s - offset, 2))
            new_end = max(new_start + 0.3, round(end_s - offset, 2))
            shifted_list.append([new_start, new_end])

            start_f = int(round(new_start * TARGET_FPS))
            end_f = int(round(new_end * TARGET_FPS))
            frame_list.append([start_f, end_f])

        shifted_seconds[stem] = shifted_list
        frame_labels[stem] = frame_list

    with open(labels_path, "w", encoding="utf-8") as f:
        json.dump(frame_labels, f, indent=2)

    print(f"  [OK] Labels written to: {labels_path}")
    print("\nShifted intervals in clean video timeline (seconds):")
```

```
    print(json.dumps(shifted_seconds, indent=4))
    print("\n[DONE] Video trimming and label synchronization complete!")


if __name__ == "__main__":
    main()
```

A.18. Мультиэнкодерная блочная экстракция аудиопризнаков (extract_russian_all_audio.py)

```
"""
Extract frame-level embeddings for Russian audio across ALL 3 encoders:
1. wav2vec2-large-xlsr-53 -> data/russian_seq_wav2vec2/ (T, 1024)
2. wavlm-base-plus        -> data/russian_seq_wavlm/    (T, 768)
3. whisper-small          -> data/russian_seq_whisper/  (T, 768)

Run: python scripts/extract_russian_all_audio.py
"""

import torch
import numpy as np
import soundfile as sf
from pathlib import Path
from transformers import (
    Wav2Vec2FeatureExtractor, Wav2Vec2Model,
    WavLMModel,
    WhisperFeatureExtractor, WhisperModel
)

SAMPLE_RATE = 16000


def extract_wav2vec2(wav_files, out_dir, device):
    out_dir.mkdir(parents=True, exist_ok=True)
    print("\n[1/3] Loading Wav2Vec2-XLSR-53...")
    model_name = "facebook/wav2vec2-large-xlsr-53"
    fe = Wav2Vec2FeatureExtractor.from_pretrained(model_name)
    model = Wav2Vec2Model.from_pretrained(model_name).to(device).eval()

    with torch.no_grad():
        for wav_path in wav_files:
            speech, _ = sf.read(str(wav_path))
            inputs = fe(speech, sampling_rate=SAMPLE_RATE, return_tensors="pt",
    padding=True)
            inputs = {k: v.to(device) for k, v in inputs.items()}
            hidden = model(**inputs).last_hidden_state.squeeze(0).cpu().numpy()
            np.save(str(out_dir / f"{wav_path.stem}.npy"), hidden.astype(np.float32))
            print(f"  [Wav2Vec2] {wav_path.stem}: {hidden.shape}")

    del model
    torch.cuda.empty_cache()


def extract_wavlm(wav_files, out_dir, device):
    out_dir.mkdir(parents=True, exist_ok=True)
    print("\n[2/3] Loading WavLM-Base-Plus...")
```

```
    model_name = "microsoft/wavlm-base-plus"
    fe = Wav2Vec2FeatureExtractor.from_pretrained(model_name)
    model = WavLMModel.from_pretrained(model_name).to(device).eval()

    with torch.no_grad():
        for wav_path in wav_files:
            speech, _ = sf.read(str(wav_path))
            inputs = fe(speech, sampling_rate=SAMPLE_RATE, return_tensors="pt",
    padding=True)
            inputs = {k: v.to(device) for k, v in inputs.items()}
            hidden = model(**inputs).last_hidden_state.squeeze(0).cpu().numpy()
            np.save(str(out_dir / f"{wav_path.stem}.npy"), hidden.astype(np.float32))
            print(f"  [WavLM]    {wav_path.stem}: {hidden.shape}")

    del model
    torch.cuda.empty_cache()


def extract_whisper(wav_files, out_dir, device):
    out_dir.mkdir(parents=True, exist_ok=True)
    print("\n[3/3] Loading Whisper-Small encoder...")
    model_name = "openai/whisper-small"
    fe = WhisperFeatureExtractor.from_pretrained(model_name)
    model = WhisperModel.from_pretrained(model_name).to(device).eval()

    chunk_samples = SAMPLE_RATE * 30  # 30-second chunks for Whisper

    with torch.no_grad():
        for wav_path in wav_files:
            speech, _ = sf.read(str(wav_path))
            chunks_emb = []

            for start in range(0, len(speech), chunk_samples):
                chunk = speech[start:start + chunk_samples]
                inputs = fe(
                    chunk, sampling_rate=SAMPLE_RATE,
                    return_tensors="pt", padding="max_length", max_length=chunk_samples
                )
                input_features = inputs.input_features.to(device)
                hidden = model.encoder(input_features).last_hidden_state

                real_frames = min(int(np.ceil(len(chunk) / 320)), hidden.shape[1])
                chunks_emb.append(hidden[0, :real_frames, :].cpu().numpy())

            full_emb = np.concatenate(chunks_emb, axis=0)
            np.save(str(out_dir / f"{wav_path.stem}.npy"), full_emb.astype(np.float32))
            print(f"  [Whisper]  {wav_path.stem}: {full_emb.shape}")

    del model
    torch.cuda.empty_cache()


def main():
    base_dir = Path(__file__).resolve().parent.parent
    audio_dir = base_dir / "data" / "russian_audio"
    wav_files = sorted(list(audio_dir.glob("*.wav")))

    if not wav_files:
        print(f"[ERROR] No .wav files found in {audio_dir}")
        return
```

```
105
106     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
107     print(f"[INFO] Found {len(wav_files)} Russian audio files. Device: {device}")
108
109     extract_wav2vec2(wav_files, base_dir / "data" / "russian_seq_wav2vec2", device)
110     extract_wavlm(wav_files, base_dir / "data" / "russian_seq_wavlm", device)
111     extract_whisper(wav_files, base_dir / "data" / "russian_seq_whisper", device)
112
113     print("\n[DONE] All 3 acoustic feature sets extracted successfully!")
114
115
116 if __name__ == "__main__":
117     main()
```